Understanding In-context Learning of Addition via Activation Subspaces

Xinyan Hu[*1], **Kayo Yin**[1], **Michael I. Jordan**[1,2], **Jacob Steinhardt**[†1], **Lijie Chen**[†1]
[1]UC Berkeley, [2]INRIA

arXiv:2505.05145v4 [cs.LG] 17 Sep 2026

Abstract

To perform few-shot learning, language models extract signals from a few input-label pairs, aggregate them into a learned prediction rule, and apply this rule to new inputs. How is this implemented in the forward pass of modern transformer models? To explore this question, we study a structured family of few-shot learning tasks for which the true prediction rule is to add an integer k to the input. We introduce a novel method that localizes the model's few-shot learning ability to only a few attention heads. This method and the findings generalize to four additional task families spanning arithmetic and semantic tasks. We then perform an in-depth analysis of individual heads via dimensionality reduction and decomposition of the heads' output spaces. For example, in Llama-3-8B-Instruct, we reduce the mechanism underlying these tasks to just three attention heads with six-dimensional subspaces, in which four dimensions track the units digit using trigonometric functions with periods 2, 5, and 10, while two dimensions track magnitude using low-frequency components. To deepen our understanding of this mechanism, we also derive a mathematical identity relating the "aggregator" and "extractor" subspaces of attention heads, allowing us to track the flow of information from individual examples to a final aggregated concept. Our results demonstrate how tracking low-dimensional subspaces of localized heads throughout a forward pass can provide insight into fine-grained computational structures in language models. Our code is available at https://github.com/xyVickyHu/addition-subspaces.

1 Introduction

Large language models (LLMs) exhibit in-context learning (ICL) abilities; for instance, they can few-shot learn new tasks from a small number of demonstrations in the prompt. To understand this ability, past works have constructed detailed models of ICL for small synthetic language models (Garg et al., 2022; Akyürek et al., 2023; von Oswald et al., 2023) as well as coarser-grained analyses of large pretrained models (Olsson et al., 2022; Hendel et al., 2023; Todd et al., 2024). However, little is known about the fine-grained computational structure of ICL for large models.

ICL extracts task information from demonstrations and applies the aggregated information to input queries. Previous work (Todd et al., 2024) constructed a vector (i.e., the "function" vector) from demonstrations that encodes the task information—for instance, adding it to the residual stream on zero-shot inputs recovers ICL behavior. However, two questions remain open: (1) How precisely do function vectors encode task information? (2) How do models aggregate information from ICL examples to form such function vectors?

To address these questions, we first develop a novel general pipeline for localizing few-shot ability to a small number of attention heads and low-dimensional subspaces, and furthermore perform a case study for arithmetic tasks (i.e., learning to add a constant k to the input) for mechanistic depth. This family of tasks has the advantage of providing a large

[*]Correspondence to: Xinyan Hu <xinyanhu@berkeley.edu>.
[†]Equal advising.

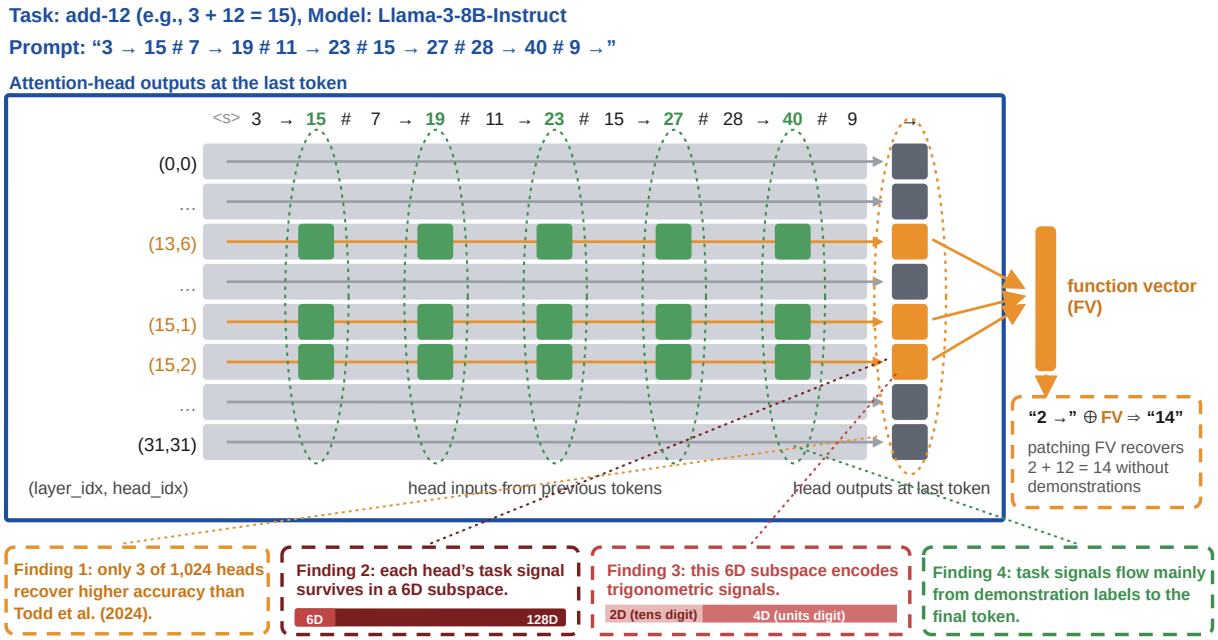


Figure 1: Key findings illustrated on Llama-3-8B-Instruct and a five-shot *add-12* prompt as an example: (1) only three attention heads carry most of the *add-k* task signal (§3); (2) each head's signal survives in a six-dimensional subspace of its outputs (§4.1); (3) that subspace trigonometrically encodes k with four dimensions for the units digit and two dimensions for the tens digit (§4.2, §4.3); and (4) signals flow mainly from demonstration labels (§5).

number of tasks (different integers k) that all share the same input domain, which is helpful to rule out domain-based shortcuts when analyzing ICL mechanisms (§2.1). Arithmetic also has the advantage of being well studied in other (non-in-context) settings (Zhou et al., 2024; Kantamneni & Tegmark, 2025), allowing us to situate our results with other known mechanisms. Finally, most LLMs perform this task reliably (e.g., 87% accuracy for Llama-3 and 90% for Qwen-2.5). Figure 1 gives an overview of our key findings on this task family.

We first introduce a head-localization pipeline that combines a novel sparse optimization objective with statistical refinements. Our approach is inspired by function vectors (Todd et al., 2024), which mimic ICL behavior when patched into the residual stream (§2.2). Whereas Todd et al. (2024) rank heads independently by their *individual* effects on ICL performance, we jointly optimize coefficients $c_h \in [0, 1]$ over all heads to identify a sparse, weighted *combination* of head outputs that maximizes intervention accuracy (§3.1). Our method successfully finds a small number of heads that recover most of the ICL performance (§3.2); for example, for addition tasks, Llama-3-8B-Instruct achieves 68% accuracy using function vectors from just three heads (78% of the original ICL accuracy). Applying the same pipeline to subtraction, multiplication, and the abstractive and extractive tasks from Todd et al. (2024) reveals shared important heads: one head in a middle layer is identified as a main head across all five families, several additional heads recur across arithmetic tasks, and broader head sets are identified for the semantic families (Appendix B).

We next perform a detailed analysis of these heads, through dimensionality reduction and decomposition. We find that the task information in the localized heads is encoded in low-dimensional subspaces across task families. For addition, which we study in depth, these subspaces exhibit an interpretable trigonometric structure. For each of the three main heads in Llama-3-8B-Instruct, we get a 6-dimensional subspace, where 4 dimensions encode periodic functions with periods 2, 5, and 10 (tracking the units digit) and 2 dimensions encode low-frequency functions (tracking the tens digit) (§4). This interestingly aligns with recent findings in non-in-context settings where addition is also encoded by trigonometric (Zhou et al., 2024) or helical functions (Kantamneni & Tegmark, 2025), suggesting a deeper relation between non-ICL and ICL machinery.

Finally, we further study the flow of information across tokens, by deriving a general mathematical relation between "extractor" and "aggregator" subspaces, building on the second-order logit lens (Gandelsman et al., 2025). This lets us study the task-related information that heads extract from each token. For example, in Llama-3-8B-Instruct we find that task information is mostly extracted from label tokens (§5.2).

In summary, we found that task-specific information in ICL can be localized to a small number of attention heads and low-dimensional subspaces with structured patterns. We also developed a way to trace the task information flowing from the "extractor" subspaces to the "aggregator" subspaces. Our findings show how even in large models, ICL mechanisms can be localized to specialized activation subspaces from a small number of heads that extract, represent, and aggregate information in interpretable ways. Our key contributions are: (1) a novel method to identify relevant attention heads for ICL; (2) a precise mathematical relation between signals from earlier tokens and the output token; and (3) a focused analysis of ICL addition that reverse-engineers rich latent structures and sophisticated computational strategies in LLMs.

2 Preliminaries

2.1 Model and Task

Model. We focus on Llama-3-8B-Instruct (Grattafiori et al., 2024) in the main body, with 32 layers, 32 attention heads per layer, and residual dimension 4096. We denote each head as a tuple (layer index, head index); both indices range from 0 to 31. To broaden our analysis in model size, training type, and model family, we also experiment on Phi-4-14B (Abdin et al., 2024) and Qwen-2.5-7B (Yang et al., 2024) and report results in the appendix. The results are directionally similar across models.

Task. We consider five families of ICL tasks: three arithmetic families (addition, subtraction, and multiplication) and the abstractive and extractive families of Todd et al. (2024), which include abstractive tasks like product-company ("iPhone 5→Apple") and extractive tasks like animal-vs-object ("bear, ukulele, blanket→bear"); Table 3 in Appendix B.1 specifies all five families. Our localization pipeline applies to each of them (Appendix B); for mechanistic depth, we center our case study on addition, a structured family of ICL tasks that we call *add-k*. For a constant k, the add-k task is to add k to a given input integer x to predict $y = x + k$. In an n-shot ICL prompt, the model is given n demonstrations of the form "$x_i \rightarrow y_i$" with $y_i = x_i + k$, concatenated using the separator "#", followed by a query "$x_q \rightarrow$" (see the top row of Figure 1 for an illustration). A key advantage of this family of tasks is that ICL prompts for different tasks only differ in k (i.e., $y_i - x_i$) but not in the input domain, enabling us to isolate task information from input content so as to dissect the ICL mechanism at a finer granularity.

This advantage motivates our focus on *add-k*: in semantic tasks such as product-company, the input domain itself leaks information about the task (Min et al., 2022), making it difficult to distinguish whether success stems from extracting the task rule or from leveraging domain-specific associations. For *add-k*, the query x_q alone provides no information about k, so the model must infer k from the demonstrations, which cleanly isolates the key components of the ICL mechanism. Moreover, addition is well studied in non-in-context settings (Zhou et al., 2024; Kantamneni & Tegmark, 2025), so studying it under ICL fills a gap and allows us to compare our findings with the known non-ICL mechanisms.

We construct data for the *add-k* task by varying $x \in [1, 100]$ and $k \in [1, 30]$ (thus $y \in [2, 130]$) since Llama-3 models can reliably solve addition in this range. We consider three types of prompts: (1) *five-shot ICL prompt*, where all five demonstrations satisfy $y_i = x_i + k$ for a fixed $k \in [1, 30]$ (also called *add-k* ICL); (2) *mixed-k ICL prompt*, where the demonstrations are $y_i = x_i + k_i$ for possibly different k_i values; and (3) *zero-shot prompt*, where there are no demonstrations: the prompt is "$x_q \rightarrow$" for some x_q. We mostly study five-shot ICL prompts and use them to generate function vectors (§2.2). We also examine signals extracted from demonstrations in mixed-k ICL prompts, a variant with mistaken demonstrations, in §5.2. We use zero-shot prompts to evaluate heads and function vectors (§2.2).

2.2 Activation Patching and Function Vectors

Next, we briefly review *activation patching*, a common interpretability technique that will be used throughout the paper, and the *function vector*, a construction that we will use to identify important heads for our ICL tasks.

Activation patching. Activation patching is performed by taking the activations of a model component when the model is run on one prompt, then patching in these activations when the model is run on a different prompt. Patching can either *replace* the model's base activations or *add to* them; we will primarily consider the latter.

Specifically, if z_l is the original value of the residual stream at layer l at the final token position "$\rightarrow$", we patch in the replacement $z_l + v$, where v is constructed from activations on a different input prompt. We choose the layer l at one-third of the network's depth and construct v from "function vector" heads, following Todd et al. (2024), as described next.

Function vectors. Function vectors v_k are vectors constructed from the outputs of chosen attention heads, designed so that adding v_k to the residual stream of a zero-shot prompt approximates the effect of the five-shot *add-k* task. For example, v_k might be the average output of one or more attention heads across a set of five-shot *add-k* examples. We define the *intervention accuracy* of v_k as the average accuracy obtained on zero-shot prompts when v_k is added to the residual stream across all tasks k. Because k is independent of the input queries, this metric captures how effectively v_k encodes task-specific information about k. For comparison, we define *clean accuracy* as the accuracy on five-shot prompts without any intervention, also averaged across all tasks k.

In more detail, consider an attention head h: let $h(p)$ denote the output of head h on prompt p at the last token position, and define h_k as the average of $h(p)$ across all five-shot *add-k* prompts.[1] Todd et al. (2024) identified a subset $\mathcal{H}$ of attention heads (for a different set of tasks) such that the vector $v_k = \sum_{h \in \mathcal{H}} h_k$ has high intervention accuracy—that is, adding it to the residual stream of zero-shot prompts effectively recovers few-shot performance.

Building on this framework, we consider multiple ways to construct v_k: (1) the task-specific mean h_k over the *add-k* task (as described above); (2) the overall mean $\overline{h}$ (i.e., average across all k), which removes task-specific information about k; or (3) the specific value $h(p)$ on an individual prompt p. Throughout the paper, we call $\overline{h}$ the *mean-ablation* of head h and call h_k the *head vector* of h with respect to k. Beyond these three choices, we sometimes project a head's output onto a lower-dimensional subspace or scale it by a coefficient. Unlike Todd et al. (2024), we recover a different set of heads by solving a novel global optimization problem (§3.1) and by systematic ablation studies (§3.2), achieving higher performance.

3 Identifying Three Aggregator Heads

To understand the mechanism behind the *add-k* ICL task, we first need to find out what model components are responsible for performing it. In this section, we find that three heads do most of the work for *add-k*, by first solving an optimization problem to select 33 heads out of 1024 (§3.1), and then narrowing down to three main heads via systematic ablations (§3.2); this process is illustrated in the first row of Figure 2. §3.3 then validates the necessity of the three main heads in the original five-shot setting.

3.1 Selecting Heads via Sparse Optimization

We will search for a set of heads that store the information for the *add-k* task, in the sense that their output activations yield good function vectors for *add-k* (§2.2).

[1] Similar to Todd et al. (2024), we approximate this average using 100 randomly generated five-shot *add-k* prompts, where each prompt contains random demonstration inputs x_i and the final query input x_q takes each integer in $[1, 100]$ exactly once.

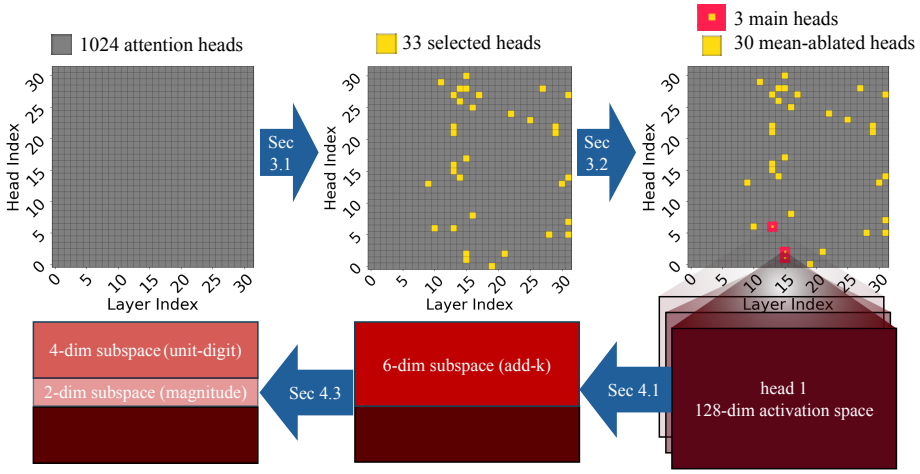


Figure 2: The chain of localization in §3 and §4. We first select 33 attention heads (out of 1024) via a global optimization method (§3.1), then narrow down to 3 main heads while mean-ablating the remaining 30 (§3.2). We next study the structure of the representation of each main head by localizing it to a six-dimensional subspace (§4.1), and decompose it into a four-dimensional subspace encoding the units digit and a two-dimensional subspace encoding the coarse magnitude (i.e., the tens digit) (§4.3).

Formulating the sparse optimization. More formally, define $v_k(c) = \sum_h c_h \cdot h_k$, the sum of head outputs weighted by c, where h goes over all 1024 heads in the model. (Recall that h_k is the output of head h averaged across a large dataset of five-shot *add-k* prompts.) We will search for a sparse coefficient vector $c \in [0,1]^{1024}$ such that adding $v_k(c)$ to the residual stream of a zero-shot prompt achieves high accuracy on the *add-k* task.

Let $\ell(x_q, y_q; v)$ be the cross-entropy loss between y_q and the model output when intervening with v on the input "$x_q \rightarrow$" (i.e., replacing z_l with $z_l + v$ in the forward pass, where z_l is the layer-l residual stream at the last token). We optimize c with respect to the loss

$$\mathcal{L}(c) = \mathbb{E}_{k,x_q}[\ell(x_q, x_q + k; v_k(c))] + \lambda \|c\|_1, \tag{1}$$

where the regularization term with weight λ promotes sparsity.

Training details. We randomly select 25 *add-k* tasks for training and in-distribution testing, using the remaining five for out-of-distribution testing. For each task, we generate 100 zero-shot prompts "$x_q \rightarrow$" with $x_q \in [1, 100]$ and target $x_q + k$. We split the 25 tasks' data into train/val/test sets in proportions 0.7/0.15/0.15. We use AdamW with learning rate 0.01, batch size 128, and regularization $\lambda = 0.05$, clipping c to $[0, 1]$ after each gradient step.

Results: the selected heads. We get coefficients that achieve high intervention accuracy. In particular, on the five held-out tasks, the function vector weighted by the final-epoch coefficients achieves an intervention accuracy of 0.812, close to the clean accuracy of 0.870. To identify important heads, we inspect the coefficient values at the end of training for each layer and head index. We find 33 heads have coefficients c_h greater than 0.2, most (21) of which are one. In contrast, the other (991) coefficients are all smaller than 0.03, most (889) of which are zero. We call these 33 heads *selected heads*, denoted $\mathcal{H}_{\text{sel}}$.

More generally, we do not hand-pick this subset: we sort the trained coefficients (averaged over the last three checkpoints to smooth end-of-training jitter) in decreasing order and cut at the largest gap between consecutive values, keeping the heads above it. This parameter-free rule recovers exactly these 33 heads for *add-k*, and is the selection rule we apply to every other task family, model, and prompt format we study (results in Appendix B).

Comparison to previous approach. We compare our optimization approach with the previous method from Todd et al. (2024) for identifying important heads, which selects heads

based on average indirect effect (AIE). To perform a fair comparison, we construct our function vector by summing the outputs of our selected heads directly without weighting by coefficients, matching their methodology. Using this construction, we achieve an intervention accuracy of 0.798, close to the clean accuracy of 0.870, indicating that our 33 heads capture most of the necessary information for the *add-k* task. In contrast, selecting the top 33 heads according to AIE yields a much lower intervention accuracy of 0.414.[2]

Intuitively, a key difference is that AIE ranks each head independently over all attention heads, whereas our optimization selects the heads jointly, accounting for their combined effects. In the next subsection, we further narrow down to an even smaller number of effective heads via ablations within this subset.

3.2 Further Narrowing to Three Heads via Ablations

Mean-ablating head subsets. We suspect that many selected heads may carry signals at the task-family level (such as ensuring the output appears as a number) without encoding the task-specific value of k. To test this, we perform *mean-ablations*: replacing a head's task-specific vector h_k with its cross-task mean $\overline{h}$ (§2.2). For a retained set of heads $\mathcal{S} \subseteq \mathcal{H}_{\text{sel}}$, we construct

$$v_k(\mathcal{S}) = \sum_{h \in \mathcal{S}} h_k + \sum_{h \in \mathcal{H}_{\text{sel}} \setminus \mathcal{S}} \overline{h}. \tag{2}$$

Mean-ablating a subset of selected heads from §3.1 gives us a meaningful task-agnostic base against which to measure the task-specific contribution of the complement $\mathcal{S}$.

Amplifying an individual head's effect for comparison. To measure the contribution of an individual selected head h, we retain and scale its task-specific vector while mean-ablating all other selected heads:

$$v_k(h, c) = c\, h_k + \sum_{g \in \mathcal{H}_{\text{sel}} \setminus \{h\}} \overline{g}, \qquad c \geq 0. \tag{3}$$

At $c = 0$, the candidate head is omitted, yielding a leave-one-out baseline; increasing $c > 0$ restores and scales its task-conditioned signal. The resulting accuracy curve therefore measures how much task-specific information that head adds on top of a reasonable base. We also intentionally increase the range of c from $[0, 1]$ in equation 1 to $\{0, \ldots, 20\}$ in equation 3 to make the task-specific effect of each head more distinguishable.

Refining head selection via two general criteria. Using equation 3, we want to identify two kinds of heads for task-specific information: (1) heads that are important at all, and (2) a core set of the most important heads.

For (1), we identify heads that have a statistically significant positive effect on accuracy, and denote them as *significant* heads. We perform an ablation comparing $c = 0$ and $c > 0$ for each selected head and run a one-sided McNemar test (McNemar, 1947) of significance. This often yields a much smaller subset than the initial set of selected heads while almost retaining their accuracy.

For (2), we are sometimes interested in finding a small set of heads that still recover a reasonable fraction of the performance. As a heuristic for finding these, we identify significant heads whose best accuracy gain over $c = 0$ is at least one quarter of the largest gain achieved by any significant head, and denote them as *main* heads. In practice, setting other thresholds or an eye test may also work, but these criteria get reliable results on our benchmarks (detailed methods and results in Appendices A and B).

Results: narrowing to three heads. For *add-k*, these criteria produce 13 significant heads and, among them, three main heads: $(15, 2)$, $(15, 1)$, and $(13, 6)$. Their accuracy curves are sharply separated from those of the rest (peak accuracies in Table 1). Keeping only

[2] Todd et al. select roughly the top 2% of heads in their work ($k = 20$ for Llama-3-8B-Instruct), which yields an even lower accuracy of 0.124 in our setting.

Head	Optimal scale c	Accuracy
$(15, 2)$	6	0.866
$(15, 1)$	6	0.817
$(13, 6)$	5	0.693
Any other selected head	per head	≤ 0.170

Table 1: Single-head intervention-accuracy peaks: the intervention accuracy of $v_k(h, c)$ from equation 3 at the head's optimal scale $c \in \{0, \ldots, 20\}$. The last row reports the maximum over the remaining 30 selected heads, each at its own optimal scale. For reference, the clean five-shot accuracy is 0.870.

these three task-specific vectors and mean-ablating the other 30 selected heads (equation 2) achieves an intervention accuracy of 0.680, 78% of the clean five-shot accuracy. We therefore focus on these three heads in the following sections.

3.3 Validating Necessity of the Three Main Heads

The experiments above show that the task-specific components of the three main heads, together with the mean-ablated base, recover much of the *add-k* signal on zero-shot prompts. We next test whether these heads are also necessary in the model's ordinary five-shot computation. Mean-ablating the three heads on randomly generated five-shot prompts sharply reduces accuracy from 0.87 to 0.43. In contrast, mean-ablating 20 random sets of three other selected heads has little effect: 95% of these sets leave accuracy at or above 0.86. Thus, the three heads do not merely carry useful signals; they are necessary for a large fraction of the model's ordinary five-shot performance.

4 Characterizing the Aggregator Subspace

For the model to perform *add-k*, it must infer the task information (the number k) from the ICL demonstrations. Our next goal is to understand how task information is represented in the activation space. Since the three aggregator heads we identified carry most of this information, we can focus on analyzing their representation space.

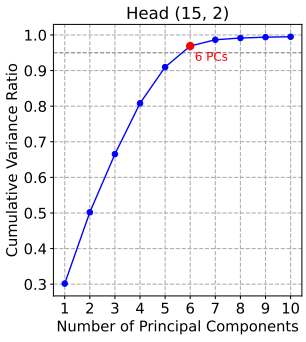
In this section, we dissect their activation spaces in three stages (illustrated in Figure 3): (1) **Localize** a six-dimensional task subspace in each head via principal component analysis (PCA) (§4.1); (2) **Rotate** this subspace into *feature directions* aligned with sinusoidal patterns across k (§4.2); (3) **Decompose** the six-dimensional space into a four-dimensional *unit-digit* subspace and a two-dimensional *magnitude* subspace that separately encode the units and tens of the task signal k (§4.3).

4.1 Localizing to Six-dimensional Subspace

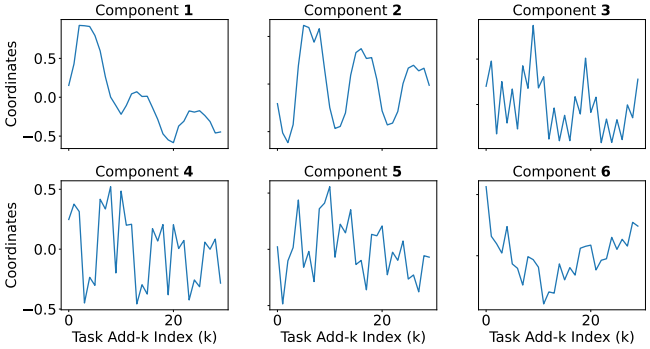
To reduce the 128-dimensional activation space per head to a more tractable space, we first perform PCA on the 30 head vectors and find that just six directions explain 97% of the task variance (Figure 3a). We then check that the function vectors found earlier remain effective after projecting onto the subspace. Specifically, we take $\mathcal{S}$ in equation 2 to be the three main heads and replace each of these head vectors h_k with its projection $\tilde{h}_k$ onto the subspace, yielding $\tilde{v}_k = \sum_{h \in \mathcal{S}} \tilde{h}_k + \sum_{h \in \mathcal{H}_{\text{sel}} \setminus \mathcal{S}} \bar{h}$. We find that $\tilde{v}_k$ has intervention accuracy 0.636, which is close to the intervention accuracy of 0.680 before projection (Tables 2 and 5). Thus, we confine our study to the concise six-dimensional subspace for each head. Both the low dimensionality and the causal sufficiency of the projection generalize across the task families, models, prompt formats, and shot counts we study (Appendix B.6).

4.2 Identifying Feature Directions Encoding Periodic Patterns

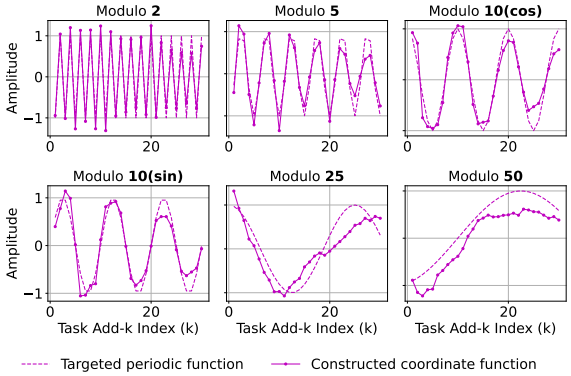
To understand how each head's six-dimensional subspace represents task information, we examine head vector coordinates (inner products with principal components (PCs)) as functions of k, revealing partially periodic patterns in the first five PCs (Figure 3b).



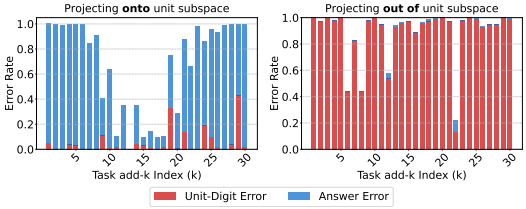
(a) **Subspace localization.** 6 PCs explain 97% of variance.



(b) **Original coordinates.** Head 1's PC coordinates as functions of k; first five PCs show partially periodic patterns.



(c) **Subspace rotation.** Linear transformation of the 6 PCs yields feature directions fitting trigonometric curves at periods $2, 5, 10, 25, 50$.



(d) **Subspace decomposition.** Projecting *onto* the unit subspace preserves unit-digit accuracy while degrading overall accuracy; projecting *out* destroys the unit-digit signal.

Figure 3: Characterizing the aggregator subspace, illustrated on head 1, $(15, 2)$ (other heads and models in Appendices C–E): (a) PCA localizes task information to a 6-d subspace; (b) PC coordinates show periodic dependence on k; (c) rotated feature directions encode clean trigonometric patterns; (d) causal steering validates the unit-digit subspace decomposition.

This motivates us to linearly transform the six PCs to find directions encoding pure periodic patterns. Mathematically, if we find a linear transformation of the six PC-coordinate functions that fits trigonometric functions, then applying the transformation to the PCs yields six directions whose coordinate functions encode the periodicity.

To find trigonometric functions to fit, we search over different periods and phases and perform least squares regression. We find six trigonometric functions at periods 2, 5, 10, 10, 25, and 50 expressible as functions of the top 6 PCs with low regression error (Figure 3c). We apply the resulting linear transformation to the six PCs to obtain directions encoding these six pure periodic patterns, which we call *feature directions*.

4.3 Decomposing into Subspaces Encoding Subsignals

Leveraging the feature directions identified previously, we decompose the head activation subspace into lower-dimensional components that separately encode different subsignals relevant to the task—in this case, the units digit and tens digit.

By construction, the coordinate function of each feature direction (viewed as a linear projection of h_k) is a periodic function of k. Mathematically, a feature direction with period T carries task information from the head vectors with "modulo T". Based on this, we hypothesize: (i) the feature direction corresponding to period two, which we call the "parity direction", encodes the parity of k in the *add-k* task; (ii) the subspace spanned by the feature

directions with periods $2, 5, 10$, which we call the "unit subspace", encodes the units digit of k; (iii) the subspace spanned by the directions with periods $25, 50$, which we call the "magnitude subspace", encodes the coarse magnitude (i.e., the tens digit) of k.

We verify these hypotheses through causal intervention. We establish (1) **sufficiency** by showing that projecting a head vector *onto* the subspace preserves the relevant task signal; and (2) **necessity** by showing that projecting a head vector *out of* the subspace (i.e., onto its orthogonal complement) destroys the relevant task signal. We show experimental results for the unit-digit subspace in Figure 3d and defer complete results to Appendix E.

5 Signal Extractors of ICL Demonstrations

Having localized most of the task signal to three heads and their six-dimensional subspaces, we now analyze *how* the model extracts it from the ICL demonstrations. We find: (1) the label tokens are the main transport channel, carrying 96–99% of each main head's task signal (§5.2); and (2) each demonstration $x_i \to y_i$ contributes a signal $y_i - x_i$ in the subspace.

5.1 Tracing the Subspace Back to Previous Tokens

We begin with a mathematical observation that lets us trace the subspace at the final token back to corresponding subspaces at earlier tokens. A head h's output at the last token of a prompt p decomposes linearly over the prompt's tokens: $h(p) = \sum_{t \in p} \alpha_t \, O_h V_h \, z_t$, where α_t is the attention weight from the last token to each token t, satisfying $\sum_{t \in p} \alpha_t = 1$, z_t is the residual stream input to the head h at token t, and V_h and O_h are the head's value and output matrices. Intuitively, the head's output is an attention-weighted sum of transformed residual streams from previous tokens, so the signal extracted from token t is $O_h V_h z_t$.

Letting W_h denote the projection onto the head's six-dimensional subspace (§4.1), the projected task signal at the final token, $W_h \, h(p)$, thus decomposes into per-token contributions $\alpha_t \, W_h O_h V_h z_t$, each lying in the image of $W_h O_h V_h$.

5.2 Signal extraction from each demonstration

To understand how demonstrations contribute to the model's generation at the last token, we identify which tokens contribute most, then examine what information they provide. By the analysis above, each token's task-signal contribution through head h is $\alpha_t \, W_h O_h V_h z_t$, which factors into (1) **extracted information:** $W_h O_h V_h z_t$, the residual stream input projected into the relevant subspace; and (2) **aggregation weight:** the attention weight α_t. On a random mixed-k ICL prompt (with possibly conflicting k_i values), both the norm of the extracted information and the aggregation weight peak at the label tokens y_i (Appendix F.1).

To quantify this on standard five-shot prompts, we project each token's contribution $\alpha_t \, O_h V_h z_t$ and the total output $h(p)$ onto the head vector h_k of the prompt's task; the ratios are signed per-token shares that sum to one by linearity. Averaged over the 25 training *add-k* tasks, the label tokens contribute 96–99% of each main head's task signal (Figure 15 in Appendix F.1), while input, separator, and query tokens contribute almost nothing.

Finally, we ask *what* information these tokens carry: for each task $k \in \{1, \ldots, 30\}$, we measure the inner product $\langle W_h O_h V_h z_t, \, \tilde{h}_k / \|\tilde{h}_k\| \rangle$ between the extracted information and the unit-normalized projected head vector of §4.1. At each label token y_i of a random mixed-k prompt, this inner product peaks at $k = y_i - x_i$ (Figure 16a, Appendix F.1): the model extracts $y_i - x_i$ from each demonstration $x_i \to y_i$.

6 Related Work

Our work builds on interpretability research that aims to uncover circuits and internal computations of language models. Many studies focus on synthetic tasks or small models (Nanda et al., 2023; Bietti et al., 2023; Reddy, 2024; Singh et al., 2024). For large pretrained LLMs, some provide coarser-grained analyses (Todd et al., 2024; Olsson et al., 2022; Hendel et al., 2023), while others target particular model families and task classes for finer-grained

insights (Hanna et al., 2023; Feng & Steinhardt, 2024; Wu et al., 2023; Zhou et al., 2024; Panickssery et al., 2024). Our work follows the latter, providing a deeper account of ICL mechanisms than prior studies.

Interpreting arithmetic tasks. A recent line of work examines how LLMs perform arithmetic (Stolfo et al., 2023; Hanna et al., 2023; Nikankin et al., 2025; Maltoni & Ferrara, 2024), and in particular addition (Nanda et al., 2023; Zhong et al., 2023; Zhou et al., 2024; Kantamneni & Tegmark, 2025). Zhou et al. (2024) and Kantamneni & Tegmark (2025) identify Fourier and helical representations for direct addition. Zhang et al. (2024b) find direct arithmetic uses fewer than 5% of attention heads focused on operands and operators. Engels et al. (2025) identify circular features for calendar-based modular arithmetic, while Levy & Geva (2025) find digit-wise circular representations of numbers. These works study how models represent numbers or execute a given operation; we instead study how a task rule is inferred from demonstrations and represented within attention heads, revealing related periodic structure in the task representation. Concurrent work identifies partial-sum representations and linear function vectors in three-operand arithmetic ICL (Polyakov et al., 2025).

Interpreting in-context learning. Researchers have constructed detailed models of ICL for small transformers in supervised learning (Garg et al., 2022; Akyürek et al., 2023; Zhang et al., 2024a; Li et al., 2023; Wu et al., 2024), as well as more complex settings (von Oswald et al., 2023; Bai et al., 2023; Bietti et al., 2023; Reddy, 2024; Guo et al., 2024; Nichani et al., 2024). For large pretrained models, ICL performance has been attributed to either induction heads (Olsson et al., 2022; Singh et al., 2024; Crosbie & Shutova, 2025; Bansal et al., 2023) or function vector (FV) heads (Todd et al., 2024; Hendel et al., 2023). Broader work traces how ICL information is encoded, aggregated, retrieved, or filtered across the prompt (Cho et al., 2025; Bakalova et al., 2025; Cho et al., 2026). Related work localizes task information with SAE features (Kharlapenko et al., 2025), while concurrent and more recent studies analyze attention heads through query-state geometry and task-recognition versus task-learning roles, respectively (Yang et al., 2025; 2026). Whereas these works focus on prompt-level information flow, feature circuits, or query transformations, Yin & Steinhardt (2025) find that few-shot ICL performance depends primarily on FV heads. Motivated by this, we study FV heads in detail, jointly identifying compact task-carrying head sets across task families—rather than ranking heads independently as in Todd et al. (2024)—and characterizing task-conditioned low-dimensional subspaces within individual head outputs. Concurrent work studies function induction in off-by-one addition (Ye et al., 2026), while subsequent work approximates FVs as combinations of example-level sub-FVs (Wang et al., 2026).

Causal analysis. Our methodological approach follows recent work on causal effects of model components via interventions on internal states (Vig et al., 2020; Geiger et al., 2021), drawing inspiration from causal mediation analysis (Todd et al., 2024), activation patching (Meng et al., 2022), and causal scrubbing (Chan et al., 2022).

7 Discussion

We provided a detailed mechanistic analysis of in-context learning for addition tasks. We found that a small number of attention heads operating in low-dimensional subspaces can extract, represent, and aggregate ICL task information in structured and interpretable ways. Our analysis proceeded in five steps: (1) sparse optimization to identify important heads (§3.1); (2) ablation to localize to a smaller subset (§3.2); (3) PCA to find low-dimensional subspaces (§4.1); (4) rotation and decomposition revealing periodic unit-digit and magnitude subspaces (§4.2, §4.3); and (5) algebraic tracing connecting aggregator subspaces at the final token to extractor subspaces at earlier y_i tokens (§5). Most steps scale easily; future work could automate the qualitative subspace examination using AI.

Beyond automation, our analysis raises two further directions. First, *within* a task family, how do the identified heads and their subspaces function individually and together? Second, *across* task families, are the important heads and subspaces shared or transferable? Appendix B.1 offers initial evidence of head-level sharing—one head, $(15, 1)$, is a main head in every family we study—but subspace-level sharing and transfer remain open.

Acknowledgments

We thank Lisa Dunlap, Alex Pan, Yaowen Ye, Ryan Wang, Prasann Singhal, Hanlin Zhu, Tony Lian, Jiaxin Ge, Shu Liu, Anya Ji, Junyi Zhang, and Baifeng Shi for their helpful feedback and suggestions on this work and earlier drafts of the manuscript. XH was supported in part by the Eric and Wendy Schmidt Fund for Strategic Innovation. KY is supported by the Vitalik Buterin Ph.D. Fellowship in AI Existential Safety. MJ is supported by the European Union (ERC-2022-SYG-OCEAN-101071601). LC is supported by a Miller Research Fellowship. JS is supported by Open Philanthropy, Simons Foundation, and NSF.

References

Marah Abdin et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.

Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? Investigations with linear models. In *Proceedings of the 2023 International Conference on Learning Representations*, 2023.

Yu Bai, Fan Chen, Huan Wang, Caiming Xiong, and Song Mei. Transformers as statisticians: Provable in-context learning with in-context algorithm selection. *Advances in Neural Information Processing Systems*, 36, 2023.

Aleksandra Bakalova, Yana Veitsman, Xinting Huang, and Michael Hahn. Contextualize-then-aggregate: Circuits for in-context learning in Gemma-2 2B. In *Second Conference on Language Modeling*, 2025.

Hritik Bansal, Karthik Gopalakrishnan, Saket Dingliwal, Sravan Bodapati, Katrin Kirchhoff, and Dan Roth. Rethinking the role of scale for in-context learning: An interpretability-based case study at 66 billion scale. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11833–11856, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.660. URL https://aclanthology.org/2023.acl-long.660/.

Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995.

Alberto Bietti, Vivien Cabannes, Diane Bouchacourt, Herve Jegou, and Leon Bottou. Birth of a transformer: A memory viewpoint. *Advances in Neural Information Processing Systems*, 36, 2023.

Lawrence Chan, Adria Garriga-Alonso, Nicholas Goldowsky-Dill, Ryan Greenblatt, Jenny Nitishinskaya, Ansh Radhakrishnan, Buck Shlegeris, and Nate Thomas. Causal scrubbing: A method for rigorously testing interpretability hypotheses. In *AI Alignment Forum*, 2022.

Hakaze Cho, Mariko Kato, Yoshihiro Sakai, and Naoya Inoue. Revisiting in-context learning inference circuit in large language models. In *Proceedings of the 2025 International Conference on Learning Representations*, 2025.

Hakaze Cho, Haolin Yang, Gouki Minegishi, and Naoya Inoue. Mechanism of task-oriented information removal in in-context learning. In *Proceedings of the 2026 International Conference on Learning Representations*, 2026.

Joy Crosbie and Ekaterina Shutova. Induction heads as an essential mechanism for pattern matching in in-context learning. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 5049–5111, 2025.

Joshua Engels, Eric J. Michaud, Isaac Liao, Wes Gurnee, and Max Tegmark. Not all language model features are one-dimensionally linear. In *Proceedings of the 2025 International Conference on Learning Representations*, 2025.

Jiahai Feng and Jacob Steinhardt. How do language models bind entities in context? In *Proceedings of the 2024 International Conference on Learning Representations*, 2024.

Yossi Gandelsman, Alexei A. Efros, and Jacob Steinhardt. Interpreting the second-order effects of neurons in CLIP. In *Proceedings of the 2025 International Conference on Learning Representations*, 2025.

Shivam Garg, Dimitris Tsipras, Percy S Liang, and Gregory Valiant. What can transformers learn in-context? A case study of simple function classes. *Advances in Neural Information Processing Systems*, 35:30583–30598, 2022.

Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 9574–9586. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/4f5c422f4d49a5a807eda27434231040-Paper.pdf.

Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Tianyu Guo, Wei Hu, Song Mei, Huan Wang, Caiming Xiong, Silvio Savarese, and Yu Bai. How do transformers learn in-context beyond simple functions? A case study on learning with representations. In *Proceedings of the 2024 International Conference on Learning Representations*, 2024.

Michael Hanna, Ollie Liu, and Alexandre Variengien. How does GPT-2 compute greater-than?: Interpreting mathematical abilities in a pre-trained language model. *Advances in Neural Information Processing Systems*, 36, 2023.

Roee Hendel, Mor Geva, and Amir Globerson. In-context learning creates task vectors. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

Subhash Kantamneni and Max Tegmark. Language models use trigonometry to do addition. *arXiv preprint arXiv:2502.00873*, 2025.

Dmitrii Kharlapenko, Stepan Shabalin, Arthur Conmy, and Neel Nanda. Scaling sparse feature circuits for studying in-context learning. In *Proceedings of the 42nd International Conference on Machine Learning*, pp. 29819–29851. PMLR, 2025.

Amit Arnold Levy and Mor Geva. Language models encode numbers using digit representations in base 10. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pp. 385–395, 2025.

Yingcong Li, Muhammed Emrullah Ildiz, Dimitris Papailiopoulos, and Samet Oymak. Transformers as algorithms: Generalization and stability in in-context learning. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19565–19594. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/li23l.html.

Davide Maltoni and Matteo Ferrara. Arithmetic with language models: From memorization to computation. *Neural Networks*, 179:106550, November 2024. ISSN 0893-6080. doi: 10.1016/j.neunet.2024.106550.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. *Advances in Neural Information Processing Systems*, 35:17359–17372, 2022.

Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11048–11064, 2022.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *Proceedings of the 2023 International Conference on Learning Representations*, 2023.

Eshaan Nichani, Alex Damian, and Jason D. Lee. How transformers learn causal structure with gradient descent. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 38018–38070. PMLR, 2024.

Yaniv Nikankin, Anja Reusch, Aaron Mueller, and Yonatan Belinkov. Arithmetic without algorithms: Language models solve math with a bag of heuristics. In *Proceedings of the 2025 International Conference on Learning Representations*, 2025.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15504–15522, 2024.

Gregory Polyakov, Christian Hepting, Carsten Eickhoff, and Seyed Ali Bahrainian. Interpretability analysis of arithmetic in-context learning in large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 1758–1777, 2025.

Gautam Reddy. The mechanistic basis of data dependence and abrupt learning in an in-context classification task. In *Proceedings of the 2024 International Conference on Learning Representations*, 2024.

Aaditya K. Singh, Ted Moskovitz, Felix Hill, Stephanie C. Y. Chan, and Andrew M. Saxe. What needs to go right for an induction head? A mechanistic study of in-context learning circuits and their formation. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 45637–45662. PMLR, 2024.

Alessandro Stolfo, Yonatan Belinkov, and Mrinmaya Sachan. A mechanistic interpretation of arithmetic reasoning in language models using causal mediation analysis. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 7035–7052, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.435. URL https://aclanthology.org/2023.emnlp-main.435/.

Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *Proceedings of the 2024 International Conference on Learning Representations*, 2024.

Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in Neural Information Processing Systems*, 33:12388–12401, 2020.

Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 35151–35174. PMLR, 2023.

Entang Wang, Yiwei Wang, Aleksandra Bakalova, and Michael Hahn. How few-shot examples add up: A causal decomposition of function vectors in in-context learning. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2605.16591.

Jingfeng Wu, Difan Zou, Zixiang Chen, Vladimir Braverman, Quanquan Gu, and Peter L. Bartlett. How many pretraining tasks are needed for in-context learning of linear regression? In *Proceedings of the 2024 International Conference on Learning Representations*, 2024.

Zhengxuan Wu, Atticus Geiger, Thomas Icard, Christopher Potts, and Noah Goodman. Interpretability at scale: Identifying causal mechanisms in Alpaca. *Advances in Neural Information Processing Systems*, 36:78205–78226, 2023.

An Yang et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Haolin Yang, Hakaze Cho, Yiqiao Zhong, and Naoya Inoue. Unifying attention heads and task vectors via hidden state geometry in in-context learning. *Advances in Neural Information Processing Systems*, 38, 2025.

Haolin Yang, Hakaze Cho, and Naoya Inoue. Localizing task recognition and task learning in in-context learning via attention head analysis. In *Proceedings of the 2026 International Conference on Learning Representations*, 2026.

Qinyuan Ye, Robin Jia, and Xiang Ren. Function induction and task generalization: An interpretability study with off-by-one addition. In *Proceedings of the 2026 International Conference on Learning Representations*, 2026.

Kayo Yin and Jacob Steinhardt. Which attention heads matter for in-context learning? In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL https://arxiv.org/abs/2502.14010.

Ruiqi Zhang, Spencer Frei, and Peter L. Bartlett. Trained transformers learn linear models in-context. *Journal of Machine Learning Research*, 25(49):1–55, 2024a.

Wei Zhang, Chaoqun Wan, Yonggang Zhang, Yiu-Ming Cheung, Xinmei Tian, Xu Shen, and Jieping Ye. Interpreting and improving large language models in arithmetic calculation. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 59932–59950. PMLR, 2024b.

Ziqian Zhong, Ziming Liu, Max Tegmark, and Jacob Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks. *Advances in Neural Information Processing Systems*, 36, 2023.

Tianyi Zhou, Deqing Fu, Vatsal Sharan, and Robin Jia. Pre-trained large language models use Fourier features to compute addition. *Advances in Neural Information Processing Systems*, 37, 2024.

A Details of the Head-Set Refinement Criteria

This appendix gives the full details of the two head-set refinement criteria stated in §3.2 — significant heads and main heads — both built on the recovery scan of equation 3. Table 2 summarizes the nested head sets they produce for Llama-3 *add-k*, together with the AIE baseline of Todd et al. (2024). We reuse both criteria for every task family, model, and prompt format we study (Appendix B).

Significant heads: statistically supporting recovery. For each selected head and each positive coefficient c in the recovery scan, we compare per-example correctness against $c = 0$ on the same prompts. Let b be the number of examples fixed by the intervention and d the number broken. We use a one-sided exact McNemar test—equivalently, an exact binomial test on the $b + d$ discordant examples (McNemar, 1947). We apply a Bonferroni correction over the positive coefficients within each head, then control the false discovery rate across heads using the Benjamini–Hochberg procedure at $q = 0.05$ (Benjamini & Hochberg, 1995). We call a head *significant* if at least one positive coefficient has $b > d$ and survives both statistical corrections.

The paired comparison accounts for variation in prompt difficulty, while the two corrections protect against selecting a head merely because we scan many coefficients and many candidates. On Llama-3 *add-k*, this procedure narrows 33 selected heads to 13 significant heads, while intervention accuracy decreases only from 0.798 to 0.774. The compression is more dramatic for Qwen-2.5-7B *add-k*: 230 selected heads narrow to 18, with accuracy 0.358 versus 0.368 for the full selected set (full results in Appendix B.2).

Main heads: materially dominant recovery. The paired tests above are designed to detect statistically reliable effects using per-example outcomes and may therefore retain heads with small overall gains. For a compact mechanistic core, we further compare each significant head's aggregated recovery gain

$$G_h = \max_{c \in \{0,\ldots,20\}} A_h(c) - A_h(0),$$

where $A_h(c)$ denotes the resulting intervention accuracy of $v_k(h, c)$ from equation 3 on zero-shot prompts. Formally, let $G^\star$ be the largest gain G_h among the significant heads. For any task family with clean accuracy at least 0.25, we call a significant head *main* if $G_h \geq G^\star/4$. On *add-k*, this rule recovers exactly the three heads of §3.2.

Beyond addition, we find one head, $(15, 1)$, in Llama-3-8B is a main head for every task family we study. Addition, subtraction, and multiplication are localized to 3, 4, and 5 main heads, respectively, with substantial reuse of the three main heads for addition. Phi-4 mirrors this structure: one head, $(18, 12)$, appears in every family's main set, and its subtraction main heads are exactly its addition main heads plus one extra head — the analogue of subtraction main heads = addition trio + $(15, 17)$ in Llama-3 (Appendices B.1, B.2). The selection is also robust along the prompt-format and shot-count axes: the three *add-k* main heads remain main heads under every prompt format and at every $n \in \{1, \ldots, 10\}$ (Appendices B.3, B.4). Moreover, in every task family, model, prompt format, and shot count we consider, the main heads achieve higher intervention accuracy than the top-k head sets selected by Todd et al. (2024)'s AIE ranking while using far fewer heads (Appendix B.5).

Comparison of the two refinements. The two criteria serve different purposes: significant-head selection retains all statistically supported contributors and aims to preserve as much of the selected set's recovery as possible, whereas main-head selection further isolates, within the significant set, a small number of materially dominant heads for subsequent mechanistic analysis.

B Generality of the Localization Pipeline

This appendix applies the full pipeline of §3 — head selection via sparse optimization, significant-head refinement, main-head selection (criteria detailed in Appendix A), and the

Head set	Selection criterion	# Heads	Accuracy [95% CI]
Selected	Identified by joint sparse optimization	33	0.798 [0.760, 0.832]
Significant	Statistically significant to fix more examples than it breaks	13	0.774 [0.736, 0.810]
Main	Accuracy gain of at least 1/4 of the best accuracy gain of significant heads	3	0.680 [0.640, 0.720]
AIE top-33	Ranked by average indirect effect	33	0.414 [0.372, 0.456]

Table 2: Nested head sets for Llama-3-8B-Instruct *add-k*, plus the AIE baseline of Todd et al. (2024) at the matched budget. Selected-set accuracy uses all 33 task-conditioned heads; the significant and main rows keep the listed set task-conditioned and mean-ablate the remaining selected heads. Brackets are 95% bootstrap confidence intervals. For reference, the clean five-shot ICL accuracy is 0.870 [0.838, 0.898] and the corrupted accuracy (zero-shot, every selected head mean-ablated) is 0.037 [0.030, 0.044].

subspace projection of §4.1 — to other task families (Table 4), models (Table 5), prompt formats (Table 7), and numbers of demonstrations (Table 8). In every variant that trains a coefficient matrix, the selected set is determined by the largest-gap rule of §3.1; the n-shot variants reuse the canonical five-shot selection.

Setup. For every variant, the family's tasks are split into training tasks and 5 held-out tasks (per-family task counts in Table 3); coefficient-matrix training and all head selection see only the training tasks, and every clean and intervention accuracy is measured on the held-out tasks (100 queries each, i.e., 500 evaluations per number). Each entry reports the mean with a 95% bootstrap confidence interval (2,000 resamples) beneath it. The rows are: **clean** (n-shot accuracy without intervention); **corrupted** (zero-shot accuracy with every selected head mean-ablated; this baseline comes from the recovery scan and is measured on the training-task scan prompts); the **intervention accuracies** of the selected, significant, and main head sets (zero-shot prompts; the function vector is the unit-coefficient sum of the row's head set, with the remaining selected heads mean-ablated), annotated with (heads used / total heads); **projected main heads** (each main head first projected onto its top principal components at 95% explained variance, annotated with PC dimensions kept / total head-output dimensions $= 128\times$ #main heads); and the **AIE baseline** (function vector from the top-k heads ranked by a paper-faithful re-implementation of Todd et al. (2024)'s average indirect effect, at the small top-k budget of their paper: $k = 20$ for Llama-3-8B and Phi-4, $k = 16$ for Qwen-2.5-7B — $\approx 2\%$ of heads for Llama-3 and Qwen). Non-arrow prompt formats use per-format retrained coefficient matrices; the n-shot variants reuse the canonical five-shot matrix and only re-run the recovery scan (20 queries per training task for $n \neq 5$). All accuracies use the exact-token-match scorer. In the tables, intervention accuracies and head budgets of our head sets are typeset in red and the AIE baseline in blue.

B.1 Task families

Table 3 specifies the five task families. Each arithmetic task fixes a constant k and applies a fixed rule to an input integer $x \in [1, 100]$, with 100 datapoints per task: the addition family is *add-k* with $k \in [1, 30]$ (§2.1), the subtraction family is *subtract-k* ($y = x - k$) with $k \in [0, 30]$ (including the identity task $k = 0$), and the multiplication family is *multiply-k* ($y = kx$) with $k \in [1, 30]$ (including the identity task $k = 1$). The abstractive and extractive families use the task datasets of Todd et al. (2024). An abstractive task maps its input to a semantically associated output that need not appear in the input: antonym and synonym; English–French and English–Spanish translation; country-capital, country-currency, landmark-country, park-country, and national-parks; person-occupation, person-sport, and person-instrument; product-company; present-past; singular-plural; sentiment; next-item and previous-item; next-capital-letter; word-length; and six capitalization/lowercasing manipulations (26 tasks in total). An extractive task outputs a span of its input. Most are word-list tasks: given a list of words, choose the first, middle, or last item; the alphabetically first or last item; or the item of the first-named category (adjective-vs-verb, verb-vs-adjective, animal-vs-object, color-vs-animal, concept-vs-object, object-vs-concept, and fruit-vs-animal — e.g., the animal

in animal-vs-object); each of these 12 rules is a separate task for lists of 3 and of 5 words. The remaining three tasks are CoNLL-2003 named-entity extraction: given a news sentence or fragment, output the (possibly multi-word) person, location, or organization it mentions (27 tasks in total).

Task family	Task family feature	# Tasks	Example task: input→label
Addition	$y = x + k, k \in [1, 30]$	30	add-7: 3 → 10
Subtraction	$y = x - k, k \in [0, 30]$	31	subtract-7: 3 → −4
Multiplication	$y = kx, k \in [1, 30]$	30	multiply-4: 3 → 12
Abstractive (Todd et al., 2024)	semantic association	26	product-company: iPhone 5→Apple
Extractive (Todd et al., 2024)	copy an input span	27	animal-vs-object: bear, ukulele, blanket→bear

Table 3: The five task families. Arithmetic inputs range over $x \in [1, 100]$ (100 datapoints per task); the abstractive and extractive task datasets come from Todd et al. (2024) and vary in size. Within each family, the split into training tasks and 5 held-out tasks is fixed once and reused across all models, prompt formats, and shot counts; every clean and intervention accuracy is measured on the held-out tasks. Example datapoints are verbatim dataset entries.

Beyond addition, we run the pipeline on subtraction, multiplication, and the extractive and abstractive families of Todd et al. (2024), all on Llama-3-8B-Instruct (Table 4). The localization structure is shared but family-dependent: one head, $(15, 1)$, is a main head in every family; the subtraction main heads are exactly the addition trio plus $(15, 17)$; multiplication is led by $(13, 6)$ (with $(15, 1)$, $(15, 17)$, $(15, 28)$, and $(13, 21)$); and the semantic families are genuinely more distributed — 11 (extractive) and 10 (abstractive) main heads with a broader layer spread — while still led by $(15, 1)$.

	Subtraction	Multiplication	Extractive	Abstractive
Clean accuracy	0.526 [0.484, 0.572]	0.348 [0.308, 0.390]	0.810 [0.776, 0.846]	0.422 [0.378, 0.466]
Corrupted accuracy	0.020 [0.015, 0.025]	0.030 [0.024, 0.037]	0.309 [0.289, 0.328]	0.023 [0.017, 0.030]
Selected heads	0.330 (28/1024) [0.290, 0.372]	0.226 (37/1024) [0.190, 0.264]	0.506 (15/1024) [0.460, 0.548]	0.300 (23/1024) [0.260, 0.342]
Significant heads	0.326 (19/1024) [0.286, 0.368]	0.134 (13/1024) [0.104, 0.162]	0.596 (13/1024) [0.554, 0.640]	0.290 (19/1024) [0.250, 0.330]
Main heads	0.290 (4/1024) [0.252, 0.330]	0.080 (5/1024) [0.058, 0.104]	0.564 (11/1024) [0.522, 0.608]	0.248 (10/1024) [0.210, 0.286]
Projected main heads	0.282 (28/512) [0.244, 0.320]	0.076 (40/640) [0.054, 0.100]	0.538 (109/1408) [0.494, 0.582]	0.218 (130/1280) [0.182, 0.256]
AIE top-k heads	0.002 (k=20) [0.000, 0.006]	0.000 (k=20) [0.000, 0.000]	0.448 (k=20) [0.406, 0.494]	0.168 (k=20) [0.136, 0.200]

Table 4: Generalization across task families (Llama-3-8B-Instruct, arrow format, five-shot). Rows as defined in the setup paragraph; brackets are 95% bootstrap confidence intervals. Parentheses report the budget of each intervention: (number of heads used / total heads in the model) for the selected, significant, and main rows; (PC dimensions kept / total dimensions of the main heads' outputs, 128 per head) for the projected row; and the top-k head budget for the AIE row. The extractive corrupted floor reflects the multiple-choice nature of that family (chance ≈ 0.31).

B.2 Models

Table 5 applies the pipeline to *add-k* on Phi-4-14B and Qwen-2.5-7B. Phi-4 localizes to three main heads $(18, 12)$, $(18, 14)$, $(17, 23)$ in layers 17–18 (of 40 layers and 1600 heads) and mir-

rors the Llama-3 family structure: $(18, 12)$ is a main head for every Phi-4 task family, and its subtraction main heads are exactly its addition main heads plus $(17, 22)$. Table 6 reports the Phi-4 task-family results, the analogue of Table 4; the per-family main-head sets are subtraction $\{(18, 12), (18, 14), (17, 22), (17, 23)\}$, multiplication $\{(18, 12), (11, 9), (18, 14), (24, 2)\}$, extractive $\{(24, 31), (19, 4), (18, 12), (21, 25), (17, 22), (17, 23)\}$, and abstractive (14 heads, again including $(18, 12)$). Note that the Phi-4 subtraction interventions from our head sets sit *above* the clean accuracy. Qwen-2.5-7B's main heads are exactly the layer-21 quartet $(21, 0)$, $(21, 5)$, $(21, 4)$, $(21, 2)$. Qwen's unit-coefficient composites carry a metric caveat: its tokenizer splits numbers into digits, which the exact-token-match scorer punishes; its trained-coefficient function vector reaches 0.460.

	Llama-3-8B-Instruct	Phi-4-14B	Qwen-2.5-7B
Clean accuracy	0.870 [0.838, 0.898]	0.928 [0.904, 0.950]	0.896 [0.870, 0.922]
Corrupted accuracy	0.037 [0.030, 0.044]	0.031 [0.024, 0.038]	0.032 [0.026, 0.039]
Selected heads	0.798 (33/1024) [0.760, 0.832]	0.908 (23/1600) [0.882, 0.932]	0.368 (230/784) [0.326, 0.410]
Significant heads	0.774 (13/1024) [0.736, 0.810]	0.910 (13/1600) [0.884, 0.934]	0.358 (18/784) [0.314, 0.400]
Main heads	0.680 (3/1024) [0.640, 0.720]	0.876 (3/1600) [0.848, 0.904]	0.270 (4/784) [0.232, 0.310]
Projected main heads	0.636 (18/384) [0.592, 0.678]	0.870 (22/384) [0.842, 0.898]	0.252 (28/512) [0.216, 0.290]
AIE top-k heads	0.124 (k=20) [0.096, 0.152]	0.728 (k=20) [0.690, 0.766]	0.136 (k=16) [0.104, 0.168]

Table 5: Generalization across models (*add-k*, arrow format, five-shot). Rows as defined in the setup paragraph; brackets are 95% bootstrap confidence intervals. Parentheses report the budget of each intervention: (number of heads used / total heads in the model) for the selected, significant, and main rows; (PC dimensions kept / total dimensions of the main heads' outputs, 128 per head) for the projected row; and the top-k head budget for the AIE row.

B.3 Prompt formats

Table 7 varies the prompt format on Llama-3-8B *add-k*, with a separately retrained coefficient matrix per format. Clean accuracy varies substantially across formats and the selected sets differ (24–46 heads), but the localization is structurally stable: **every format recovers the three main heads** $(15, 2)$**,** $(15, 1)$**,** $(13, 6)$ (formats qa and ab additionally recover $(15, 17)$, giving four main heads there).

B.4 Number of demonstrations

Table 8 varies the number of demonstrations n from 1 to 10 while reusing the canonical five-shot coefficient matrix. Clean accuracy rises from 0.256 to 0.944 (saturating around $n \approx 7$), while the intervention accuracies plateau from $n \approx 4$. The localization is again structurally stable: **every** n **recovers the same three main heads**. The AIE baseline, by contrast, depends strongly on the demonstration budget (it ranks heads by corrupting demonstration labels): at n=1 its top-33 list omits $(15, 2)$ and $(13, 6)$ entirely, and its top-20 accuracy climbs from 0.014 to ~ 0.3 only at large n.

B.5 Comparison with the AIE baseline

The AIE rows above evaluate Todd et al. (2024)'s head selection at the small top-k budget of their paper. The comparison is not driven by the budget: at matched k equal to our

	Subtraction	Multiplication	Extractive	Abstractive
Clean accuracy	0.658 [0.618, 0.700]	0.426 [0.382, 0.468]	0.768 [0.732, 0.808]	0.442 [0.398, 0.486]
Corrupted accuracy	0.036 [0.029, 0.043]	0.030 [0.024, 0.036]	0.261 [0.243, 0.279]	0.033 [0.026, 0.040]
Selected heads	0.754 (37/1600) [0.718, 0.792]	0.438 (36/1600) [0.392, 0.482]	0.412 (18/1600) [0.370, 0.456]	0.326 (31/1600) [0.286, 0.366]
Significant heads	0.776 (13/1600) [0.740, 0.814]	0.438 (24/1600) [0.392, 0.482]	0.416 (16/1600) [0.372, 0.460]	0.324 (29/1600) [0.284, 0.364]
Main heads	0.754 (4/1600) [0.718, 0.792]	0.128 (4/1600) [0.102, 0.158]	0.402 (6/1600) [0.360, 0.444]	0.234 (14/1600) [0.198, 0.272]
Projected main heads	0.754 (30/512) [0.718, 0.792]	0.120 (38/512) [0.094, 0.148]	0.390 (63/768) [0.348, 0.432]	0.182 (187/1792) [0.148, 0.216]
AIE top-k heads	0.038 (k=20) [0.022, 0.056]	0.004 (k=20) [0.000, 0.010]	0.344 (k=20) [0.302, 0.388]	0.180 (k=20) [0.146, 0.214]

Table 6: Generalization across task families for Phi-4-14B (arrow format, five-shot). Rows as defined in the setup paragraph; brackets are 95% bootstrap confidence intervals. Parentheses report the budget of each intervention: (number of heads used / total heads in the model) for the selected, significant, and main rows; (PC dimensions kept / total dimensions of the main heads' outputs, 128 per head) for the projected row; and the top-k head budget for the AIE row.

	arrow (orig.)	qa	io	ab	textlabel	fx
Clean accuracy	0.870 [0.838, 0.898]	0.418 [0.372, 0.462]	0.532 [0.490, 0.574]	0.700 [0.660, 0.738]	0.832 [0.798, 0.864]	0.348 [0.306, 0.390]
Corrupted accuracy	0.037 [0.030, 0.044]	0.018 [0.013, 0.023]	0.034 [0.026, 0.041]	0.030 [0.024, 0.037]	0.018 [0.013, 0.024]	0.032 [0.025, 0.039]
Selected heads	0.798 (33/1024) [0.760, 0.832]	0.498 (28/1024) [0.454, 0.542]	0.470 (46/1024) [0.426, 0.512]	0.736 (38/1024) [0.696, 0.772]	0.582 (24/1024) [0.540, 0.624]	0.270 (32/1024) [0.230, 0.310]
Significant heads	0.774 (13/1024) [0.736, 0.810]	0.492 (26/1024) [0.448, 0.536]	0.440 (9/1024) [0.396, 0.482]	0.726 (15/1024) [0.686, 0.762]	0.558 (21/1024) [0.516, 0.598]	0.270 (11/1024) [0.230, 0.308]
Main heads	0.680 (3/1024) [0.640, 0.720]	0.432 (4/1024) [0.388, 0.476]	0.334 (3/1024) [0.294, 0.374]	0.650 (4/1024) [0.608, 0.690]	0.496 (3/1024) [0.452, 0.538]	0.190 (3/1024) [0.156, 0.224]
Projected main heads	0.636 (18/384) [0.592, 0.678]	0.410 (24/512) [0.366, 0.456]	0.306 (18/384) [0.266, 0.348]	0.614 (24/512) [0.570, 0.656]	0.464 (18/384) [0.418, 0.506]	0.184 (18/384) [0.150, 0.218]
AIE top-k heads (k=20)	0.124 [0.096, 0.152]	0.126 [0.096, 0.156]	0.078 [0.056, 0.102]	0.196 [0.160, 0.230]	0.104 [0.078, 0.130]	0.064 [0.042, 0.086]

Table 7: Generalization across prompt formats (Llama-3-8B-Instruct *add-k*, five-shot; per-format retrained coefficient matrices). Rows as defined in the setup paragraph; brackets are 95% bootstrap confidence intervals. Parentheses report the budget of each intervention: (number of heads used / total heads in the model) for the selected, significant, and main rows; (PC dimensions kept / total dimensions of the main heads' outputs, 128 per head) for the projected row; and the top-k head budget for the AIE row.

selected-set size, the AIE top-k set also trails our selected set in every variant — on canonical *add-k*, 0.414 [0.372, 0.456] at k=33 versus our 0.798 [0.760, 0.832]. In **every** task family, model, prompt format, and shot count above, our 3–14 main heads achieve higher intervention accuracy than the AIE top-k set while using fewer heads; the pattern also reproduces on the Phi-4 non-addition families (Table 6). We note one nuance in fairness to AIE: its per-head *ranking* is informative — it places our main heads at or near the top of the per-head list in essentially every variant — but its top-k *sets* underperform because the tail of the ranking does not sum into an effective function vector.

n	1	2	3	4	5 (orig.)	6	7	8	9	10
Clean accuracy	0.256 [0.216, 0.294]	0.502 [0.458, 0.544]	0.700 [0.658, 0.740]	0.816 [0.780, 0.850]	0.870 [0.838, 0.898]	0.898 [0.870, 0.924]	0.932 [0.908, 0.954]	0.944 [0.924, 0.962]	0.940 [0.918, 0.960]	0.944 [0.924, 0.962]
Corrupted accuracy	0.046 [0.030, 0.066]	0.034 [0.020, 0.050]	0.028 [0.014, 0.044]	0.032 [0.018, 0.048]	0.037 [0.030, 0.044]	0.032 [0.018, 0.048]	0.034 [0.020, 0.050]	0.032 [0.018, 0.048]	0.032 [0.018, 0.048]	0.032 [0.018, 0.048]
Selected heads (33/1024)	0.188 [0.154, 0.222]	0.586 [0.544, 0.630]	0.752 [0.714, 0.790]	0.782 [0.746, 0.818]	0.798 [0.760, 0.832]	0.774 [0.736, 0.808]	0.772 [0.736, 0.808]	0.770 [0.734, 0.806]	0.772 [0.736, 0.806]	0.754 [0.716, 0.790]
Significant heads	0.184 (6) [0.150, 0.218]	0.534 (8) [0.490, 0.578]	0.700 (10) [0.658, 0.740]	0.776 (10) [0.738, 0.812]	0.774 (13) [0.736, 0.810]	0.756 (11) [0.716, 0.792]	0.746 (10) [0.706, 0.782]	0.750 (10) [0.712, 0.786]	0.748 (10) [0.710, 0.784]	0.740 (10) [0.702, 0.778]
Main heads (3/1024)	0.156 [0.122, 0.186]	0.452 [0.408, 0.496]	0.626 [0.582, 0.668]	0.674 [0.632, 0.716]	0.680 [0.640, 0.720]	0.664 [0.622, 0.704]	0.652 [0.608, 0.694]	0.652 [0.608, 0.692]	0.644 [0.600, 0.684]	0.628 [0.584, 0.670]
Projected main heads (18/384)	0.156 [0.122, 0.186]	0.410 [0.366, 0.452]	0.592 [0.548, 0.636]	0.638 [0.594, 0.680]	0.636 [0.592, 0.678]	0.632 [0.588, 0.674]	0.618 [0.572, 0.660]	0.616 [0.570, 0.658]	0.620 [0.574, 0.662]	0.620 [0.574, 0.662]
AIE top-k heads (k=20)	0.014 [0.004, 0.026]	0.042 [0.026, 0.060]	0.072 [0.050, 0.096]	0.074 [0.052, 0.098]	0.124 [0.096, 0.152]	0.312 [0.270, 0.350]	0.186 [0.152, 0.220]	0.318 [0.276, 0.356]	0.172 [0.138, 0.204]	0.316 [0.274, 0.354]

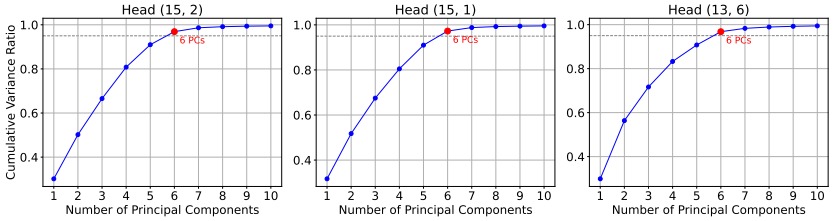
Table 8: Generalization across the number of demonstrations n (Llama-3-8B-Instruct *add-k*, arrow format; the five-shot coefficient matrix is reused, so the selected set is fixed at 33 heads). Significant-head counts are per-n (out of 1024). Recovery scans for $n \neq 5$ use 20 queries per training task. Brackets are 95% bootstrap confidence intervals. Parenthetical row-label annotations report the budget of each intervention: (number of heads used / total heads in the model) for the selected and main rows; (PC dimensions kept / total dimensions of the main heads' outputs, 128 per head) for the projected row; and the top-k head budget for the AIE row.

B.6 Subspace dimensionality across variants

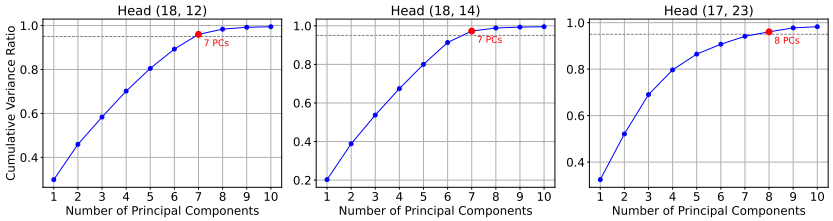
The six-dimensional finding of §4.1 generalizes as low-dimensional structure with family-dependent dimensionality. The number of principal components needed to reach 95% of task variance for main heads is 6 per head on Llama-3 addition (every format and every n), 7 on subtraction, 6–9 on multiplication, 7–8 on Phi-4 addition, and 5–8 on Qwen-2.5 addition, rising to 6–11 (extractive) and 11–15 (abstractive) for the semantic families. Causal sufficiency is universal: as the projected rows of Tables 4–8 show, replacing every main head's vector by its projection onto these subspaces costs at most 0.052 accuracy in any displayed cell (0.058 across the full grid including significant sets).

C Supplement for §4.1

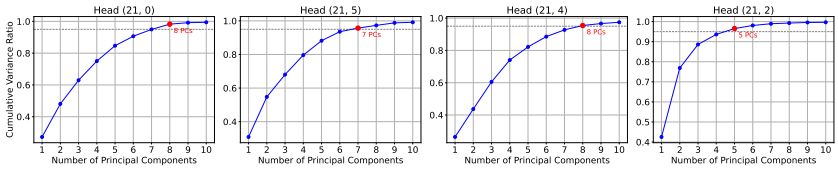
We perform PCA on the 30 head vectors of each main head and plot the cumulative explained variance ratio against the number of PCs (Figure 4). Six directions explain 97% of the task variance for every Llama-3-8B-Instruct main head (Figure 4a); the Phi-4 main heads reach 95% at 7–8 PCs and the Qwen-2.5-7B main heads at 5–8 PCs (Figures 4b and 4c).



(a) Llama-3-8B-Instruct



(b) Phi-4-14B



(c) Qwen-2.5-7B

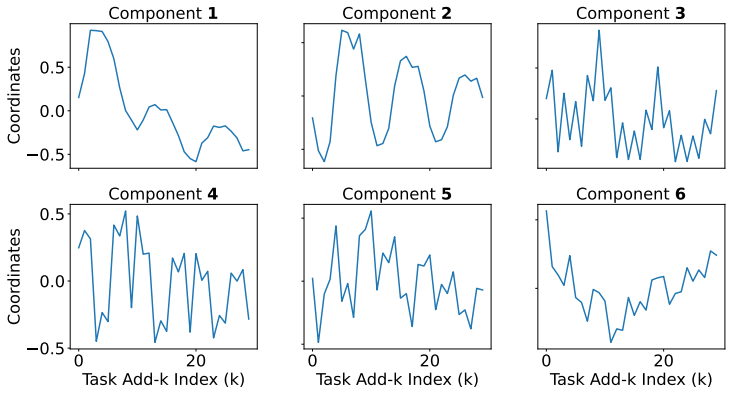
Figure 4: Cumulative explained variance ratio vs. number of PCs for each main head. The first six PCs capture 97% of the task variance for the Llama-3-8B-Instruct main heads; the Phi-4 main heads reach 95% at 7–8 PCs and the Qwen-2.5-7B main heads at 5–8 PCs.

D Supplement for §4.2

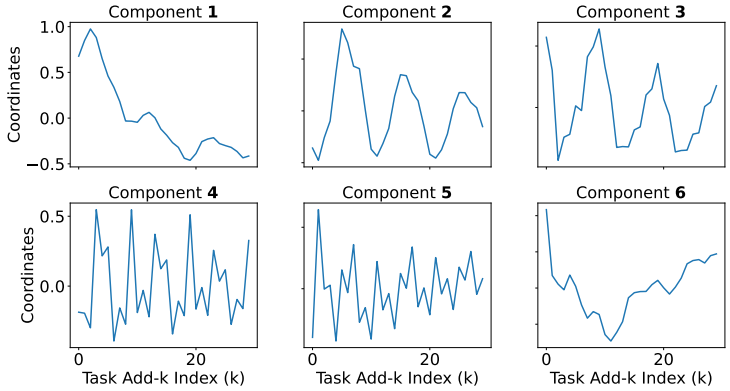
We first present additional figures for Llama-3-8B-Instruct, followed by results for Phi-4 and Qwen-2.5-7B. The periodic structure generalizes: all Phi-4 main heads fit the six trigonometric patterns well, while for Qwen-2.5 the periodic (unit-digit) patterns are fit well by heads $(21, 0)$ and $(21, 4)$ but poorly by $(21, 2)$ and $(21, 5)$, whose coordinates instead follow the two low-frequency (magnitude) directions and behave distinctly across the intervals $[1, 10]$, $[11, 20]$, and $[21, 30]$. We conjecture this difference arises from tokenization: Qwen encodes numbers digit by digit, leading to discontinuities across each 10-interval, whereas Llama-3 and Phi-4 represent every number in our range as a single token.

D.1 Additional figures for §4.2

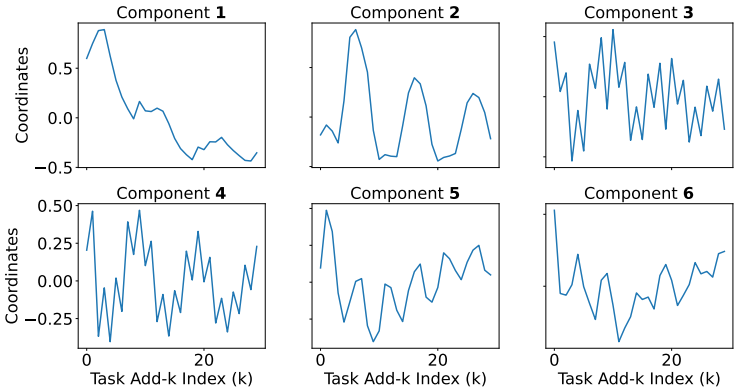
In §4.2, we found six trigonometric functions that can be linearly fitted by the coordinate functions. Here we first supplement the plot from which we observe the periodic pattern of the coordinate functions for the three heads (Figure 5) and then show the fitting functions for the three heads (Figure 6).



(a) Head $(15, 2)$

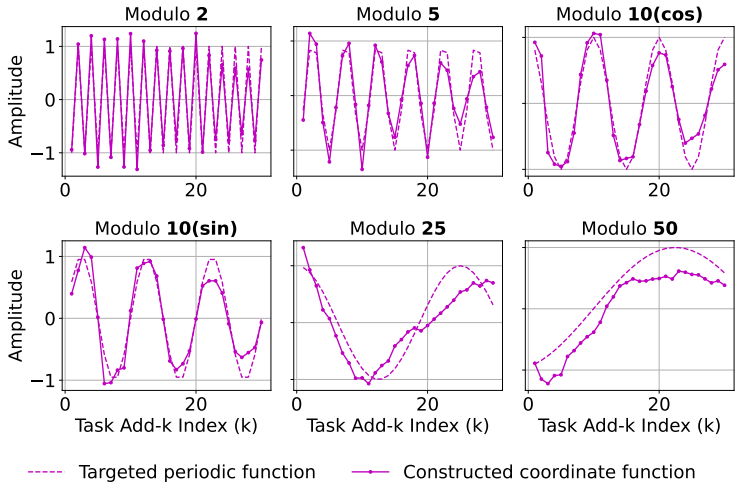


(b) Head $(15, 1)$

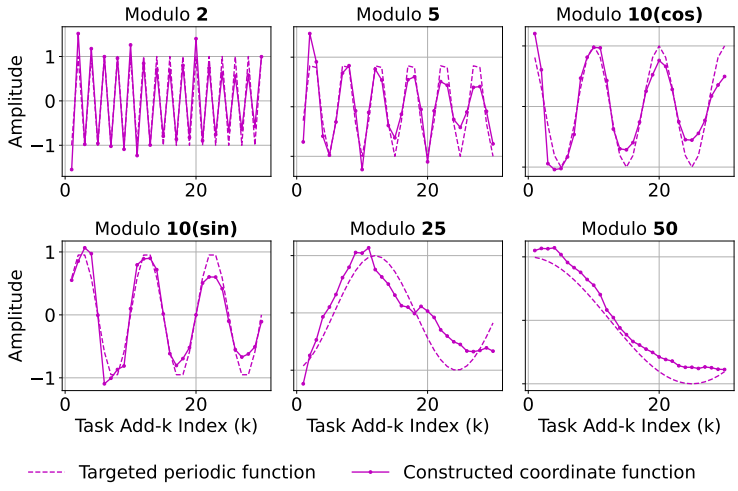


(c) Head $(13, 6)$

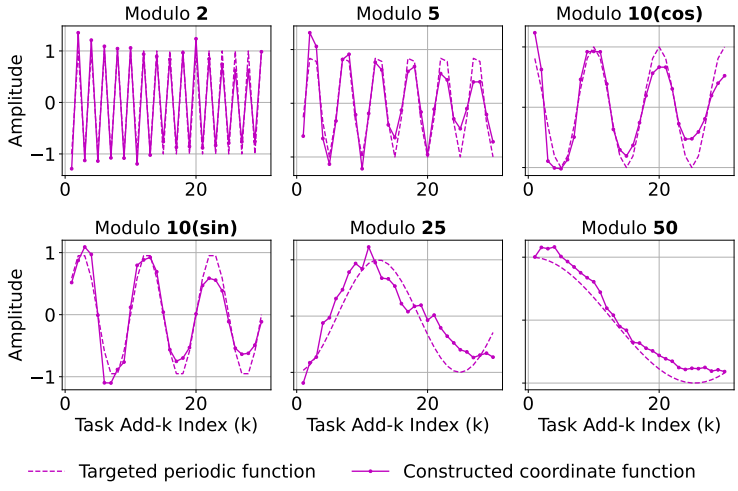
Figure 5: Coordinates of three heads' vectors (inner products with PCs) for the first six PCs across different add-k tasks on Llama-3-8B-Instruct. Periodic patterns are visible in the first few PCs of each head.



(a) Head $(15, 2)$



(b) Head $(15, 1)$

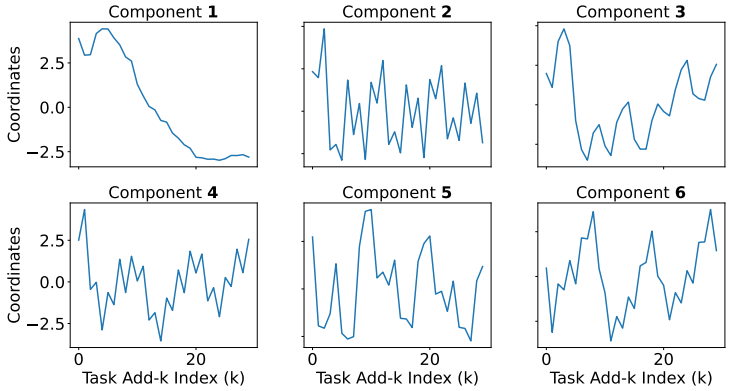


(c) Head $(13, 6)$

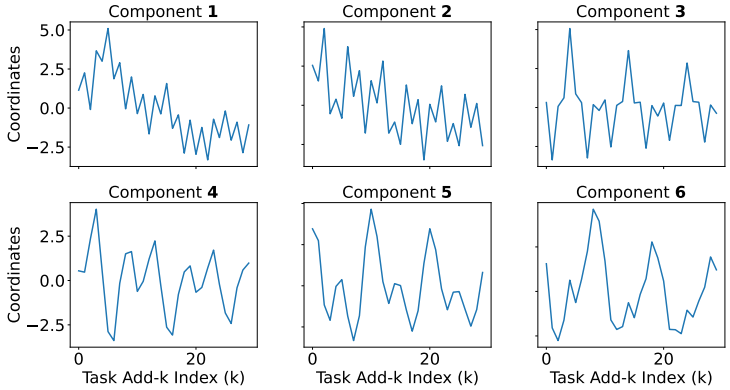
Figure 6: Coordinate functions of three heads can fit six trigonometric functions with periods $2, 5, 10, 10, 25$, and 50 on Llama-3-8B-Instruct.

D.2 Phi-4-14B for §4.2

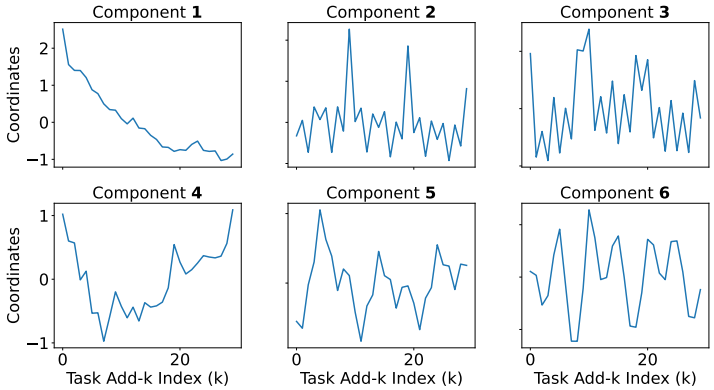
The Phi-4 main heads show similar periodic structure to Llama-3-8B-Instruct: their PC coordinates are periodic in k (Figure 7) and fit the six trigonometric patterns well (Figure 8).



(a) Head $(18, 12)$

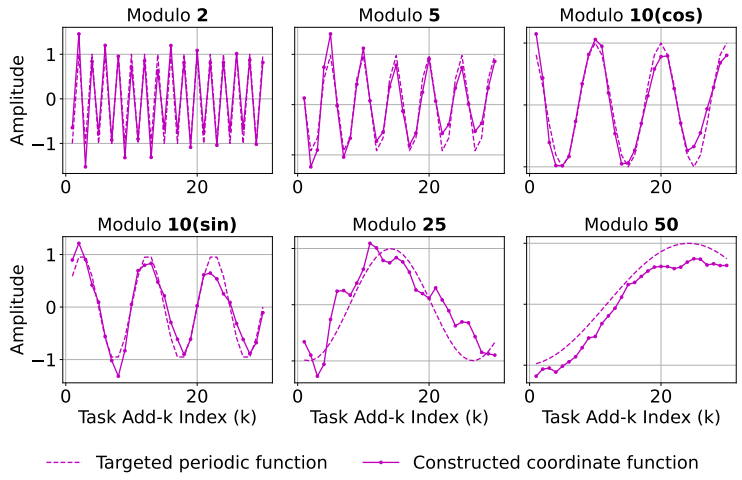


(b) Head $(18, 14)$

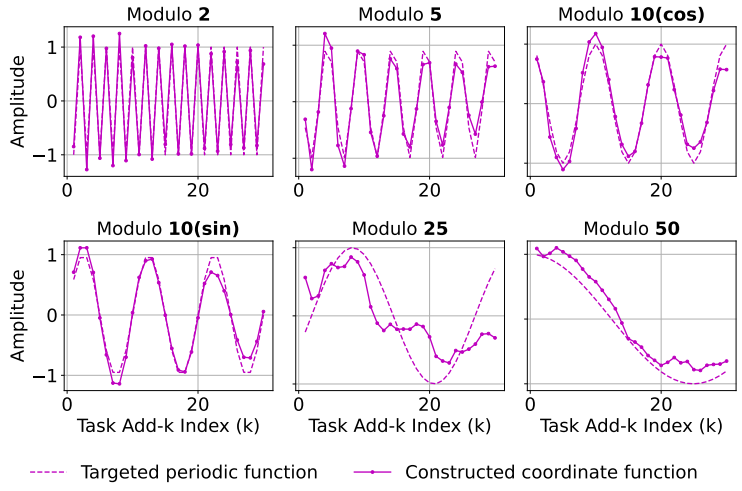


(c) Head $(17, 23)$

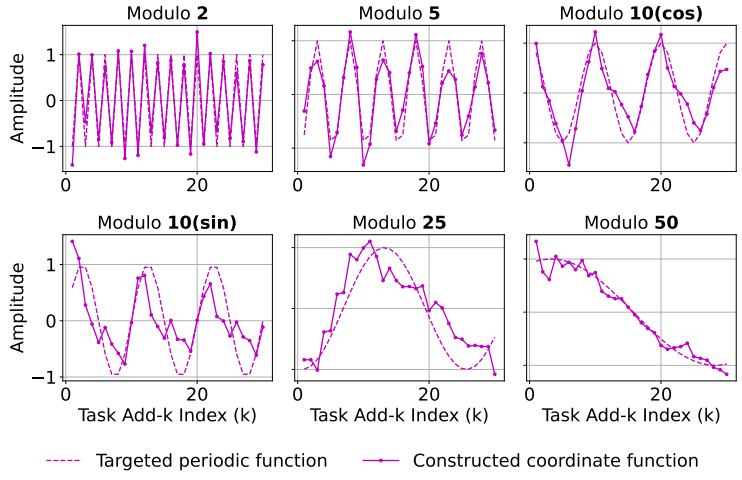
Figure 7: Coordinates of the three Phi-4 main heads' vectors (inner products with PCs) for the first six PCs across different add-k tasks. Periodic patterns are visible in the first few PCs of each head.



(a) Head $(18, 12)$



(b) Head $(18, 14)$

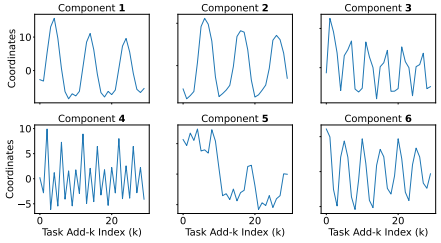


(c) Head $(17, 23)$

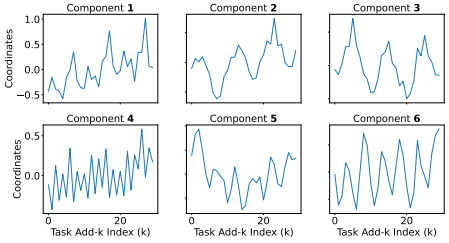
Figure 8: Coordinate functions of the three Phi-4 main heads fit six trigonometric functions with periods $2, 5, 10, 10, 25$, and 50.

D.3 Qwen-2.5-7B for §4.2

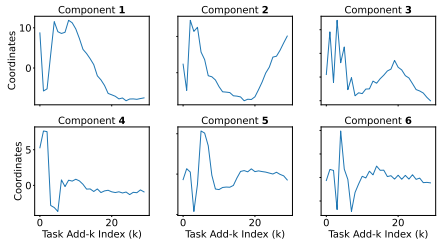
Heads $(21, 0)$ and $(21, 4)$ show periodic patterns similar to the Llama-3 heads, while heads $(21, 2)$ and $(21, 5)$ do not encode the unit-digit periodicity; their coordinates instead follow the low-frequency magnitude directions and behave distinctly across the intervals $[1, 10]$, $[11, 20]$, and $[21, 30]$ (Figures 9 and 10). We conjecture this difference arises from tokenization: Qwen encodes numbers digit by digit, creating discontinuities across each 10-interval.



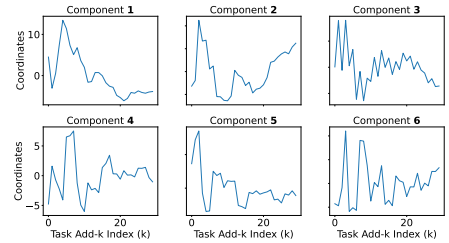
(a) Head $(21, 0)$



(b) Head $(21, 4)$

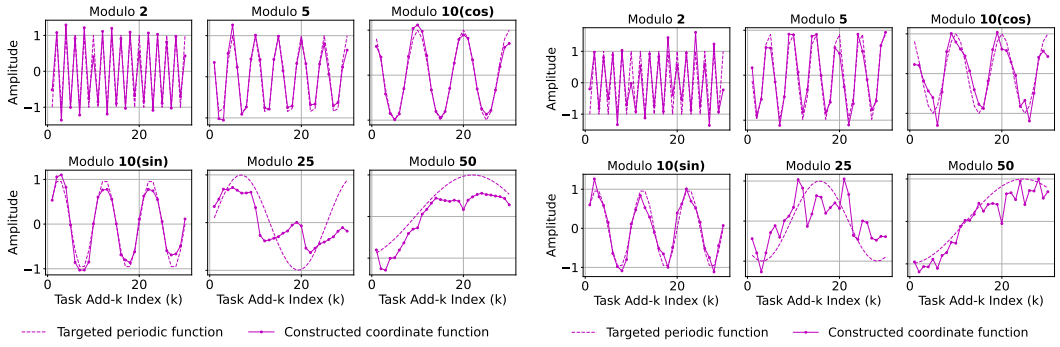


(c) Head $(21, 2)$



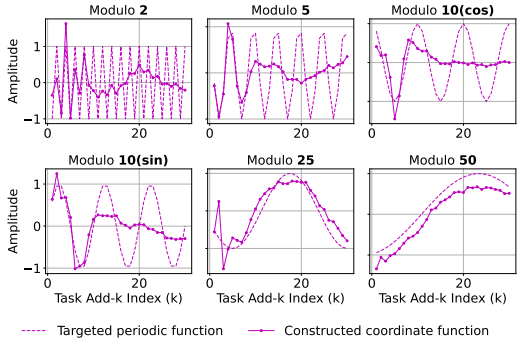
(d) Head $(21, 5)$

Figure 9: Coordinates of the four Qwen-2.5-7B main heads' vectors (inner products with PCs) for the first six PCs across different add-k tasks. Heads $(21, 0)$ and $(21, 4)$ reveal periodic patterns, whereas heads $(21, 2)$ and $(21, 5)$ show discontinuous behaviors across the intervals $[1, 10]$, $[11, 20]$, and $[21, 30]$.

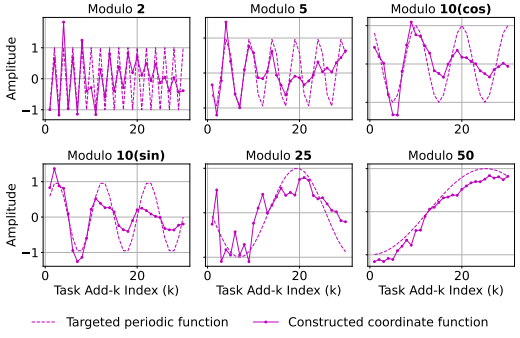


(a) Head $(21, 0)$

(b) Head $(21, 4)$



(c) Head $(21, 2)$



(d) Head $(21, 5)$

Figure 10: Six trigonometric functions with periods $2, 5, 10, 10, 25$, and 50 fitted by the coordinate functions of the four Qwen-2.5-7B main heads. Heads $(21, 0)$ and $(21, 4)$ fit the periodic functions well, while heads $(21, 2)$ and $(21, 5)$ fit only the low-frequency directions, consistent with their non-periodic coordinate behaviors.

E Supplement for §4.3

Recall that the three hypotheses in §4.3 are as follows:

(i) the feature direction corresponding to period two, which we call the "parity direction", encodes the parity of k in the *add-k* task;

(ii) the subspace spanned by the feature directions with periods $2, 5, 10$, which we call the "unit subspace", encodes the units digit of k;

(iii) the subspace spanned by the directions with periods $25, 50$, which we call the "magnitude subspace", encodes the coarse magnitude (i.e., the tens digit) of k.

We first show the experimental results validating hypothesis (ii) for head 1 (Figure 11), and then show analogous results for hypotheses (i) and (iii) and for the other two heads (Figures 12–14). Projecting out of the parity direction does not substantially increase errors for either the parity or the final answer across all tasks, perhaps because parity is easy to obtain (e.g., random choice leads to 0.5 accuracy) or parity is redundantly encoded elsewhere in the six-dimensional subspace.

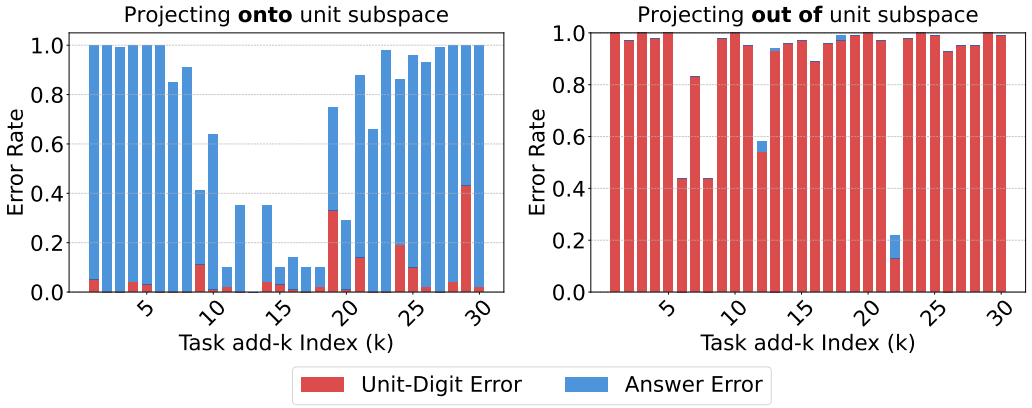
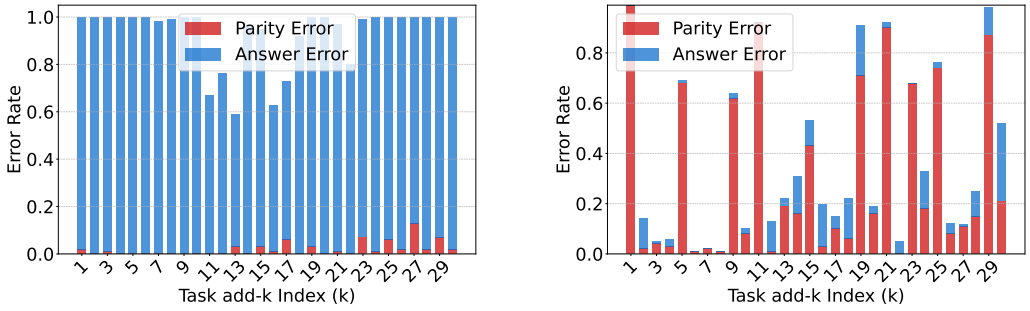
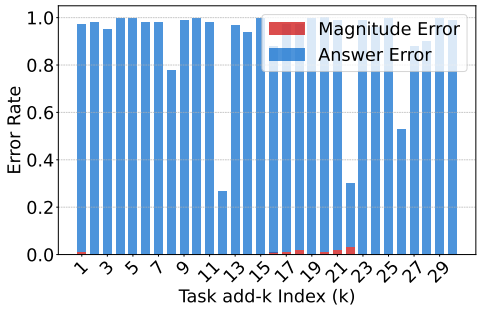


Figure 11: The error rates for the units digit and the final answer across tasks when projecting head 1's vectors **onto** (left) and **out of** (right) the "unit subspace". Projecting onto the unit subspace yields a low unit-digit error rate even when the final-answer error remains high, while projecting out leads to high unit-digit error rates that almost fully account for final-answer errors. This confirms that the unit subspace encodes the unit-digit signal.

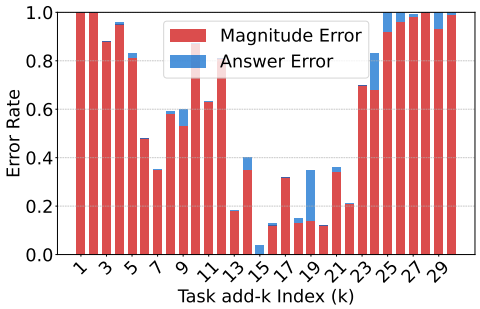


(a) Onto parity direction

(b) Out of parity direction

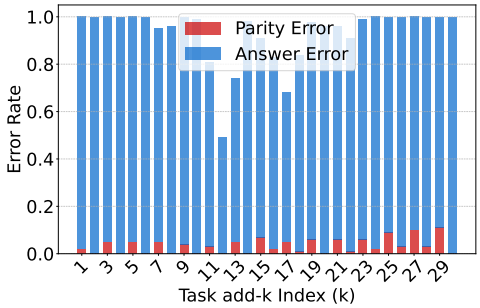


(c) Onto magnitude subspace

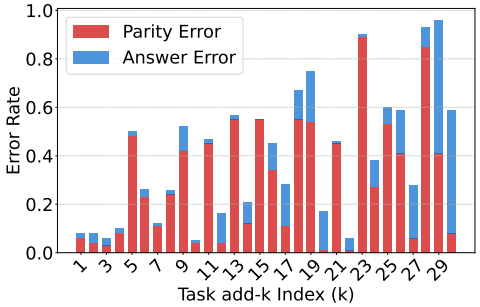


(d) Out of magnitude subspace

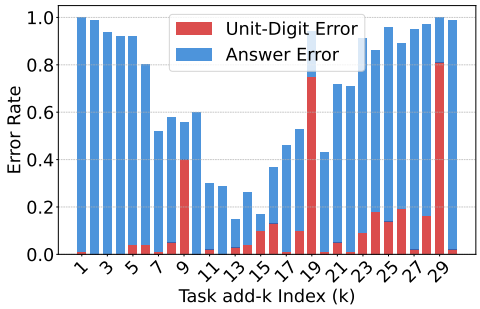
Figure 12: Validation of hypotheses (i) and (iii) for head 1. Each row shows results for the parity and magnitude subspaces (left: projection **onto**; right: projection **out of**). The unit-digit hypothesis (ii) is omitted here since it is already shown in Figure 11.



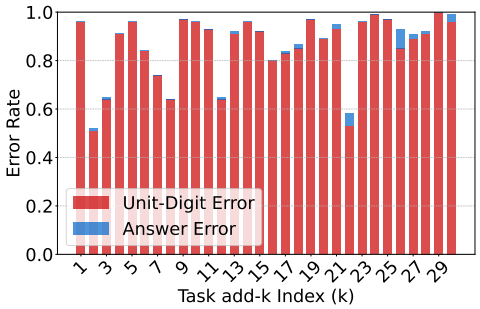
(a) Head 2 onto parity direction



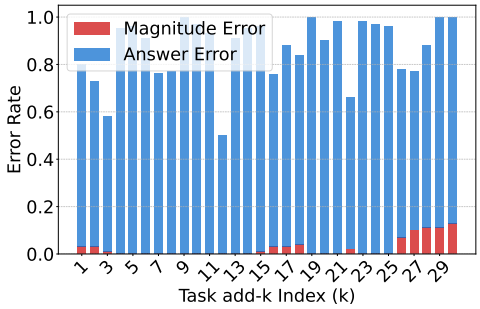
(b) Head 2 out of parity direction



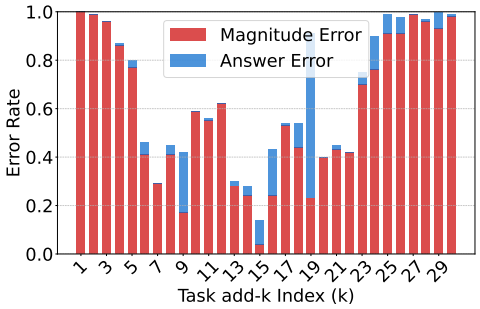
(c) Head 2 onto unit subspace



(d) Head 2 out of unit subspace

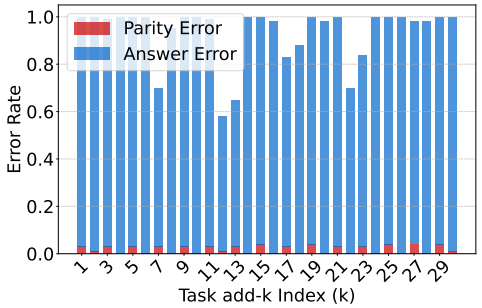


(e) Head 2 onto magnitude subspace

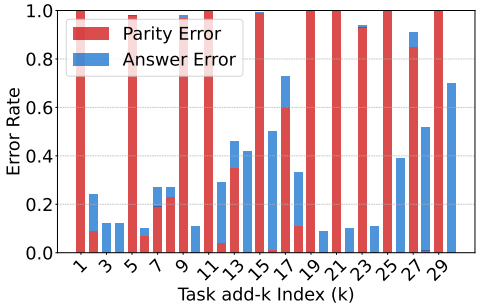


(f) Head 2 out of magnitude subspace

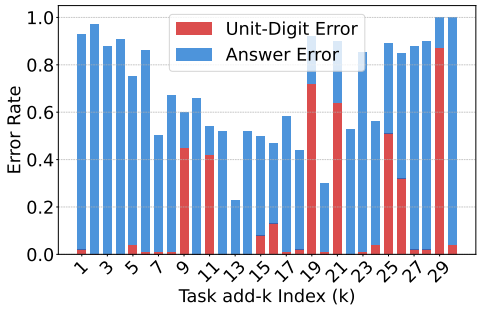
Figure 13: Validation of hypotheses (i)–(iii) for head 2. Each row shows the projection effects for the parity, unit, and magnitude subspaces, respectively.



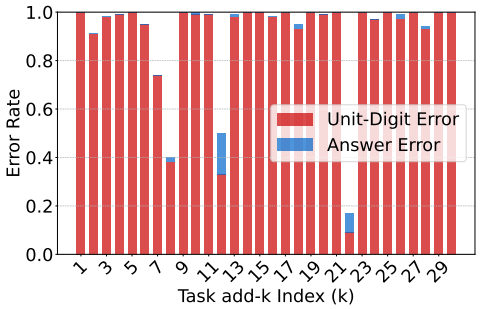
(a) Head 3 onto parity direction



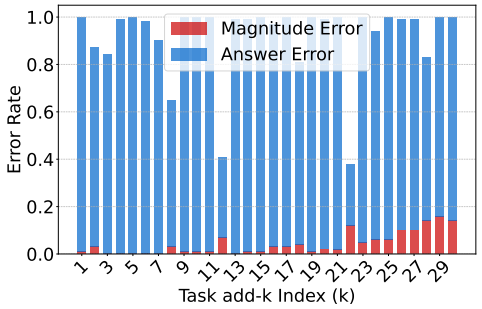
(b) Head 3 out of parity direction



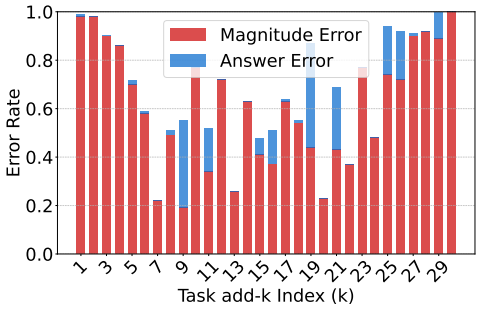
(c) Head 3 onto unit subspace



(d) Head 3 out of unit subspace



(e) Head 3 onto magnitude subspace



(f) Head 3 out of magnitude subspace

Figure 14: Validation of hypotheses (i)–(iii) for head 3. The evidence is slightly weaker than for heads 1 and 2, consistent with head 3's lower intervention accuracy (§3.2).

F Supplement for §5.2

This appendix provides the figures supporting §5.2.

F.1 Additional figures for §5.2

First, Figure 15 reports the per-position decomposition of each main head's task signal on standard five-shot prompts.

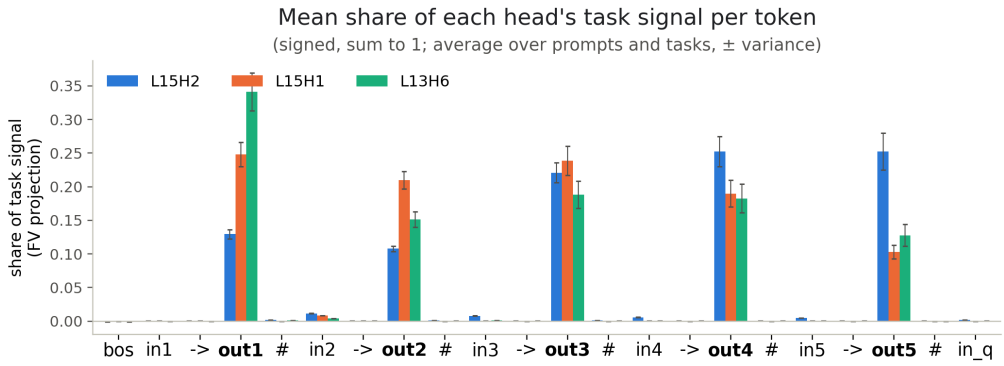
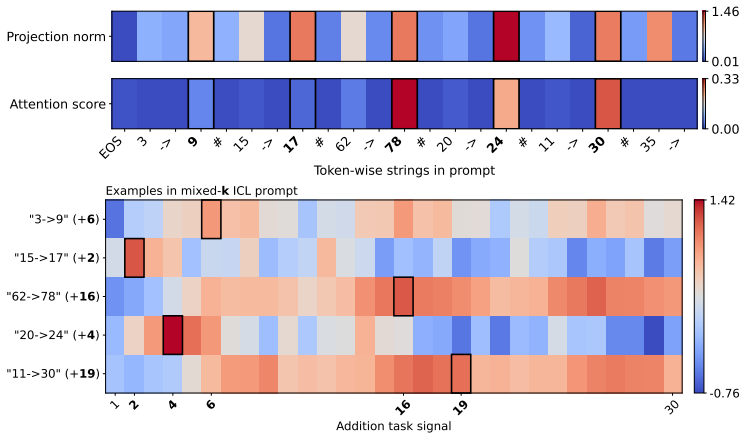
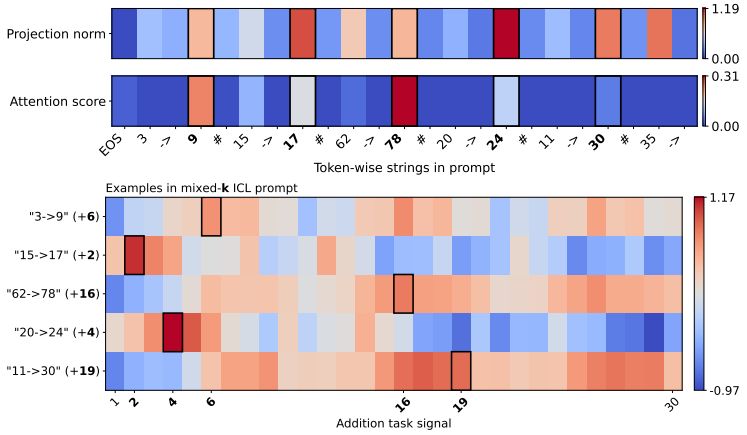


Figure 15: Mean share of each main head's task signal contributed by each token position on five-shot *add-k* prompts (signed shares sum to one; averaged over prompts and the 25 training tasks; error bars show variance). The demonstration label tokens y_i (bold) carry essentially the entire task signal for all three main heads.

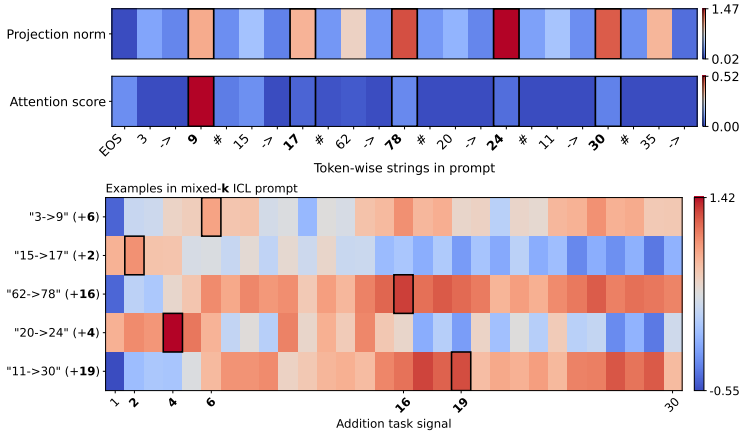
Next, we turn to the signals extracted from each token of a mixed-k ICL prompt ("3→9 # 15→17 # 62→78 # 20→24 # 11→30 # 35→"). All three heads behave similarly: the extracted signal is strongest at the y_i tokens, and its direction encodes $y_i - x_i$ (Figure 16).



(a) Head $(15, 2)$



(b) Head $(15, 1)$



(c) Head $(13, 6)$

Figure 16: Llama-3-8B-Instruct. For each head, the **top panel** shows the two factors of each previous token's task-signal contribution $\alpha_t W_h O_h V_h z_t$: (1) the norm of the extracted information $\|W_h O_h V_h z_t\|$ and (2) the attention weight α_t; both consistently peak at the tokens $t = y_i$ (in bold). The **bottom panel** shows the inner product between the projected signal from each y_i and the head vector $\tilde{h}_k$ (normalized to unit norm), which peaks at $k = y_i - x_i$, indicating that each head extracts $y_i - x_i$ from its corresponding demonstration $x_i \to y_i$.